

운영체제 2

남궁명수

[1-1] 가상화: 주소 공간의 개념	2
[1] 초기 시스템	2
[2] 멀티프로그래밍과 시분할	2
[3] 주소 공간	4
[4] 목표	6
[1-2] 가상화: 메모리 관리 API	8
[1] 메모리 공간의 종류	8
[2] malloc() 함수	10
[3] free() 함수	12
[4] 흔한 오류	12
[5] 운영체제의 자원	18
[6] 기타 함수들	19
[1-2] 가상화: 주소 변환의 원리	20
[1] 가정	23
[2] 사례	24
[3] 동적(하드웨어 기반)재배치	26
[4] 하드웨어 지원: 요약	30
[5] 운영체제 이슈	33
[1-3] 가상화: 세그멘테이션	34
[1] 세그멘테이션: 베이스/바운드의 일반화	34
[2] 세그먼트 종류의 파악	35
[4] 공유 자원	39
[5] 소단위 대 대단위 세그멘테이션	44
[6] 운영체제의 지원	49
[1-4] 가상화: 빈 공간 관리	53
[1] 가정	54
[2] 저수준 기법들	56
[3] 기본 전략	62
[4] 다른 접근법	65

[1-1] 가상화: 주소 공간의 개념

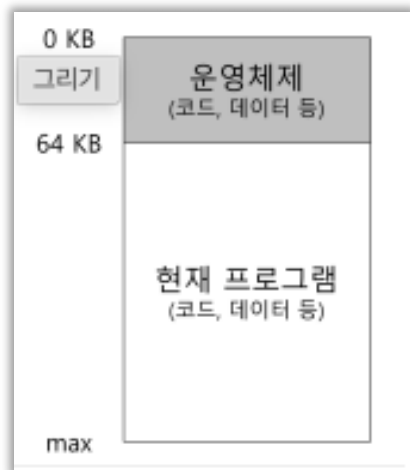
핵심 질문: 메모리를 어떻게 가상화하는가

운영체제는 물리 메모리를 공유하는 다수의 프로세스에게 어떻게 프로세스 전용의 커다란 주소 공간이라는 개념을 제공할 수 있는가?

[1] 초기 시스템

메모리 관점에서 초기 컴퓨터는 사용자에게 제공되는 기능이 매우 제한적이었다.

컴퓨터의 물리 메모리는 아래 그림에서 보는 것과 같이 구성되어 있었다.



운영체제는 메모리(이 예에서는 물리 주소 0 부터)에 상주하는 루틴들, 즉 라이브러리 집합을 관리하였다. 프로그램(프로세스)은 하나만 메모리에 존재했으며, 예를 들어, 물리 주소 64K 부터 시작하여 나머지 메모리를 사용하였다.

당시에는 특별한 가상화 기능이 거의 존재하지 않았고, 사용자도 운영체제로부터 많은 것을 기대하지 않았다. 따라서 운영체제 개발자의 환경은 상대적으로 단순하고 쉬운 편이었다.

[2] 멀티프로그래밍과 시분할

시간이 흐르면서 컴퓨터는 고가 장비였기 때문에, 사람들이 컴퓨터를 더 효과적으로 공유할 필요가 생겼다.

이로 인해 멀티프로그래밍 시대가 도래하였다.

여러 프로세스가 실행 준비 상태에 있고, 운영체제는 이들을 전환하면서 실행하였다.

예를 들어, 한 프로세스가 입출력을 수행하면, CPU 는 다른 프로세스로 전환되었다.

이러한 전환은 CPU 이용률을 증가시켰다.

당시 컴퓨터의 가격은 수십만 또는 수백만 달러에 달했기 때문에, 효율성 개선이 특히 중요했다.

곧 사용자 수가 늘어나면서 시분할 시대가 시작되었다.

많은 사용자가 동시에 컴퓨터를 사용하였고, 실행 중인 작업으로부터 즉각적인 응답을 원했기 때문에 대화식 이용의 필요성이 커졌다.

또한, 일괄 처리의 한계와 긴 프로그램-디버그 사이클에 진친 프로그래머들도 시분할의 필요성을 느꼈다.

시분할을 구현하는 한 가지 방법은, 한 프로세스를 짧은 시간 동안 실행시키고, 그 기간 동안 프로세스에게 모든 메모리 접근 권한을 부여하는 것이다.



실행이 끝나면 프로세스를 중단하고, 중단 시점의 상태를 디스크 등 장치에 저장한 뒤, 다른 프로세스의 상태를 메모리에 올려 다시 짧은 시간 동안 실행하는 방식이었다.

하지만 이 방법에는 큰 문제가 있었다. 너무 느리게 동작하며, 특히 메모리가 커질수록 속도가 더욱 느려졌다.

레지스터 상태 저장과 복원은 빠르지만, 메모리 전체 내용을 디스크에 저장하는 것은 매우 시간이 걸렸다.

해결책은 프로세스 전환 시 프로세스를 메모리에 그대로 유지하면서, 운영체제가 시분할을 효율적으로 구현하도록 하는 것이다.

위 그림에는 세 개의 프로세스(A, B, C)가 있고, 각 프로세스는 512KB 물리 메모리에서 각각 작은 부분을 할당받았다.

하나의 CPU 를 가정하면, 운영체제는 실행할 한 프로세스(A)를 선택하고, 나머지 프로세스(B 와 C)는 준비 큐에서 실행을 기다린다.

시분할 시스템이 대중화되면서 운영체제는 새로운 요구 사항이 생겼다.

여러 프로그램이 메모리에 동시에 존재할 경우 보호(protection)가 매우 중요한 문제가 된다. 한 프로세스가 다른 프로세스의 메모리를 읽거나, 더 나아가 변경할 수 있는 상황은 결코 허용되어서는 안된다.

[3] 주소 공간

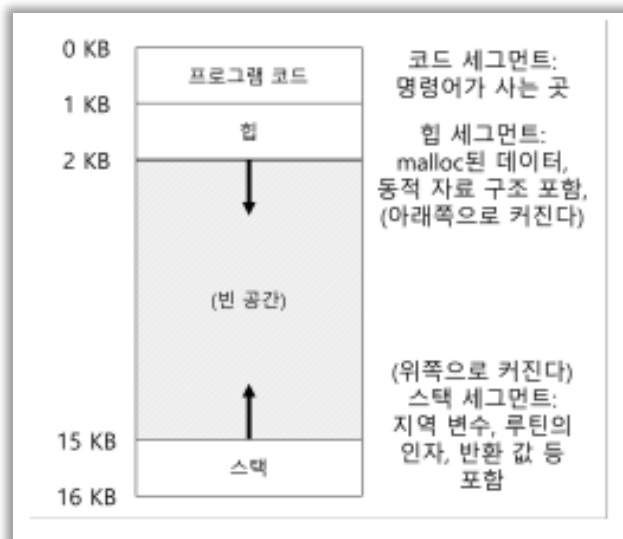
운영체제는 프로그램이 다른 프로세스의 메모리에 접근하거나 변경하지 못하도록 보호할 책임이 있다.

이를 위해 사용자가 쉽게 쓸 수 있는 메모리 개념, 즉 주소 공간(address space)을 제공한다. 주소 공간은 실행 중인 프로그램이 가정하는 메모리의 모습을 나타내며, 운영체제가 메모리를 가상화하는 핵심개념이다.

주소 공간에는 프로그램의 모든 메모리 상태가 포함된다. 예를 들어:

- 코드(code): 프로그램 명령어를 저장하며, 반드시 메모리에 존재해야 한다.
- 스택(stack): 함수 호출 위치, 지역 변수, 함수 인자, 반환 값을 저장
- 힙(heap): 동적 메모리 할당용. C의 malloc(), C++/Java의 new 등을 통해 메모리를 할당한다.

그 외에도 정적으로 초기화된 변수 등 다른 구성 요소가 있지만, 여기서는 코드, 스택, 힙 세 가지로 단순화하여 설명한다.



위 그림은 작은 주소 공간(16KB)을 보여준다.

- 프로그램 코드는 주소 공간 상단에 위치하며, 예제에서는 주소 0부터 1KB를 차지한다. 코드가 정적이므로 추가 메모리 할당이 필요 없고, 주소 공간 상단에 배치된다.
- 실행과 함께 크기가 변할 수 있는 두 영역, 힙과 스택, 는 주소 공간 양 끝에 배치된다.
 - 힙: 코드 뒤 1KB에 시작하여 아래 방향으로 확장(사용자가 malloc() 호출 시)
 - 스택: 주소 공간 상단 16KB에서 시작하여 위 방향으로 확장(프로시저 호출 시)

이러한 배치는 관례일 뿐이며, 필요에 따라 다른 방식으로 주소 공간을 배치할 수 있다. 다만, 여러 쓰레드가 공존할 때는 이렇게 나누면 동작하지 않을 수 있다.

주소 공간은 운영체제가 실행 중인 프로그램에게 제공하는 추상화이다.

실제로 프로그램이 물리 주소 0~16KB 에 존재하는 것은 아니며, 임의의 물리 주소에 탑재된다. [2] 멀티프로그래밍과 시분할에서 나온 그림에서 프로세스 A, B, C 를 보면, 각 프로세스가 서로 다른 주소에 탑재된 것을 알 수 있다.

운영체제가 메모리를 이렇게 관리하는 것을 가상화(virtualizing memory)라고 부른다. 즉, 프로그램은 자신이 특정 주소(예: 0)에서 시작하고 매우 큰 주소 공간(예: 32 비트 또는 64 비트)을 갖고 있다 생각하지만, 실제 물리 주소는 다르다.

예를 들어, 프로세스 A 가 주소 0(가상 주소)에서 load 연산을 수행하면, 운영체제는 하드웨어의 도움을 받아 물리 주소 320KB(A 가 실제 탑재된 위치)를 읽도록 보장한다. 이 과정이 바로 메모리 가상화의 핵심이며, 현대 컴퓨터 시스템의 기저를 이루는 중요한 기술이다.

[4] 목표

이제 운영체제의 메모리 가상화 기능에 대해 논의할 차례이다.

운영체제는 단순히 메모리를 가상화하는 것을 넘어서, 이 가상화를 효율적으로 안전하게 수행해야 한다.

이를 위해 가상 메모리 시스템(VM)이 달성해야 하는 몇 가지 목표를 살펴보자.

1. 투명성(Transparency)

가상 메모리 시스템의 첫 번째 목표는 투명성이다.

- 실행 중인 프로그램은 메모리가 가상화되었다는 사실을 인식하지 못해야 한다.
- 프로그램은 자신만의 전용 물리 메모리를 가진 것처럼 행동하며, 실제로 여러 프로세스가 메모리를 공유하더라도 운영체제와 하드웨어가 이를 무대 뒤에서 처리한다.

2. 효율성(Efficiency)

두 번째 목표는 효율성이다.

- 시간 효율: 가상화로 인해 프로그램 실행 속도가 지나치게 느려지지 않아야 한다.
- 공간 효율: 가상화를 지원하기 위해 너무 많은 메모리를 사용해서는 안된다.
- 운영체제는 TLB(Translation Lookaside Buffer)와 같은 하드웨어 지원 기능을 활용하여 시간-공간 효율적인 가상화를 구현한다.

3. 보호(Protection)

세 번째 목표는 보호이다.

- 운영체제는 각 프로세스를 다른 프로세스로부터, 그리고 운영체제 자신을 프로세스로부터 보호해야 한다.
- 프로세스가 자신의 작업을 수행할 때, 다른 프로세스나 운영체제 메모리에 접근하거나 영향을 미치지 못하도록 해야 한다.
- 이를 통해 각 프로세스는 고립된 환경에서 안전하게 실행될 수 있다.

다음 장에서는 운영체제와 하드웨어 지원을 포함한 메모리 가상화 기법을 자세히 탐구한다. 또한, 빈 공간 관리, 페이지 교체 정책 등 가상 메모리 운영체제 정책에 대해서도 학습하며, 현대 가상 메모리 시스템이 실제로 어떻게 동작하는 이해하게 될 것이다.

팁: 고립(isolation)의 원칙

고립은 신뢰할 수 있는 시스템을 구축하는 핵심 원칙이다.

- 서로 적절하게 고립된 두 엔티티(프로세스)는, 한쪽이 실패하더라도 다른 쪽에 영향을 주지 않는다.
- 운영체제는 프로세스 간 고립을 유지함으로써, 한 프로세스가 다른 프로세스에 피해를 주는 것을 방지한다.
- 또한, 메모리 고립을 활용하여 프로그램이 운영체제 자체의 동작에 영향을 미치지 못하도록 보장한다.

현대 운영체제에서는 고립 개념을 확장하여 구성 요소 간 고립도 구현한다.

- 예를 들어, 마이크로커널 구조는 전통적인 모놀리식 커널보다 높은 신뢰성과 안정성을 제공할 수 있다.

여담: 당신이 보는 모든 주소는 가상 주소이다.

C 프로그램에서 포인터를 출력한 적이 있는가?

- 우리가 보는 값(보통 16 진수로 표시되는 큰 수)은 가상 주소(virtual address)이다.
- 프로그램 코드가 메모리에 탑재된 정확한 물리 주소를 출력할 수는 없다.
- 사용자 프로그램이 접근할 수 있는 주소는 모두 가상 주소이며, 실제 물리 주소를 아는 것은 오직 운영체제와 하드웨어 뿐이다.

즉, 프로그램에서 주소를 출력하면 그 주소는 메모리에 배치에 대한 환상일 뿐이며, 실제 위치는 운영체제와 하드웨어가 관리한다.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main(int argc, char *argv[]) {  
    printf("location of code : %p\n", (void *) main);    // 코드 영역  
    printf("location of heap : %p\n", (void *) malloc(1)); // 힙 영역  
    int x = 3;  
    printf("location of stack : %p\n", (void *) &x);    // 스택 영역  
    return x;  
}
```

64 비트 Mac OS X 에서 실행하면 출력 예시는 다음과 같다.

location of code : 0x1095afe50

location of heap : 0x1096008c0

location of stack : 0x7fff691aea64

- 주소 공간에서 코드가 먼저 등장하고, 그 다음 힙, 마지막으로 스택이 가상 공간의 반대편에 위치한다.
- 모든 주소는 가상 주소이며, 운영체제와 하드웨어가 물리 주소로 변환하여 실제 메모리 접근을 수행한다.

[1-2] 가상화: 메모리 관리 API

핵심 질문: 어떻게 메모리를 할당하고 관리해야 하는가

Unix/C 프로그램에서 메모리를 할당하고 관리하는 방법을 이해하는 것은 강력하고 안정적인 소프트웨어를 구축하는 데 중요하다.

일반적으로 어떤 인터페이스가 사용되는가?

어떤 실수를 해서는 안 되는가?

[1] 메모리 공간의 종류

C 프로그램이 실행되면 두 가지 유형의 메모리 공간이 사용된다.

첫 번째는 스택(stack) 메모리이다.

스택 메모리는 메모리의 할당과 반환이 컴파일러에 의해 자동으로 처리된다.

이러한 이유로 스택 메모리는 자동 메모리(automatic memory)라고도 불린다.

C 프로그램에서 스택 메모리를 사용하는 것은 매우 간단하다.

예를 들어, func()라는 함수 안에서 x 라는 정수를 저장할 공간이 필요하다고 하자.

다음과 같이 선언하면 된다.

```
void func(){
    int x; // 스택에 int 형 변수 할당
    ...
}
```

이 경우 컴파일러가 나머지 작업을 수행한다.

func() 함수가 호출되면 스택에 해당 변수의 공간이 자동으로 확보되며, 함수가 종료될 때 컴파일러가 자동으로 메모리를 반환한다.

따라서 함수 호출 이후에도 유지되어야 하는 데이터는 스택에 저장하지 않는 것이 좋다.

두 번째 유형의 메모리는 힙(heap) 메모리이다.

힙메모리는 할당과 반환을 프로그래머가 직접 관리해야 한다.

즉, 메모리 할당과 해제에 대한 책임이 전적으로 프로그래머에게 있다.

예를 들어, 힙에 정수 하나를 할당하려면 다음과 같이 작성할 수 있다.

```
void func() {  
    int *x = (int *)malloc(sizeof(int));  
}
```

이 코드에서는 한 줄에서 스택과 힙 할당이 동시에 발생한다.

1. 컴파일러는 `int *x` 선언을 보고 정수 포인터를 위한 공간을 스택에 할당한다.
2. 프로그램은 `malloc()`을 호출하여 정수를 저장할 공간을 힙에서 요청한다.
3. `malloc()`은 메모리의 주소를 반환하며(실패 시 `NULL` 반환)
4. 반환된 주소는 스택에 있는 포인터 변수 `x`에 저장되어 프로그램에서 사용된다.

이처럼 힙 메모리는 명시적으로 관리해야 하며 다양한 방식으로 사용되기 때문에, 사용자와 시스템 모두에게 부담이 될 수 있다.

실제로 많은 프로그램 오류가 힙 메모리 관리에서 발생한다.

[2] malloc() 함수

malloc()함수의 사용 방법은 매우 간단하다.

힙(heap)에 할당할 메모리 공간의 크기를 인자로 전달하면, 할당이 성공한 경우 새로 할당된 공간의 포인터를 반환하고, 실패한 경우 NULL 을 반환한다.

malloc()을 사용하기 위해서는 다음과 같이 stdlib.h 헤더 파일을 포함해야 한다.

```
#include <stdlib.h>
void *malloc(size_t size);
```

이 정보를 통해 malloc()을 사용하려면 소스 코드에 stdlib.h 를 포함시키면 된다는 것을 알 수 있다.

사실 C 프로그램은 기본적으로 C 라이브러리와 링크되기 때문에 헤더 파일 없이도 동작할 수 있지만, 헤더 파일을 포함하면 컴파일러가 malloc() 호출을 올바르게 사용했는지 검사할 수 있다.

예를 들어, 전달된 인자의 개수나 데이터 타입이 올바른지 확인할 수 있다.

malloc()의 인자

malloc()의 인자는 size_t 타입이며, 요청할 메모리 크기를 바이트 단위로 나타낸다.

대부분의 프로그래머는 크기를 직접 숫자로 입력하지 않는다.

예를 들어, 10 과 같은 숫자를 직접 사용하는 방식은 권장되지 않는다.

대신 sizeof() 연산자를 사용하는 것이 일반적이다.

예를 들어, double 타입의 값을 저장할 공간을 할당하려면 다음과 같이 작성한다.

```
double *d = (double *)malloc(sizeof(double));
```

여기서 sizeof()는 C 언어에서 컴파일 시간에 계산되는 연산자이다.

예를 들어, sizeof(double)은 컴파일 과정에서 실제 크기(일반적인 8 바이트)로 대체되어 malloc()에 전달된다.

따라서 sizeof()는 함수 호출이 아니라 연산자로 간주된다.

sizeof() 사용 시 주의할 점

sizeof()는 데이터 타입뿐만 아니라 변수 이름을 인자로 사용할 수도 있다.
하지만 기대한 결과가 나오지 않을 수도 있으므로 주의해야 한다.

예를 들어, 다음 코드를 살펴보자.

```
int *x = malloc(10 * sizeof(int));  
printf("%dWn", sizeof(x));
```

첫 번째 줄에서는 정수 10 개를 저장할 수 있는 공간을 할당하였다.

하지만 두 번째 줄에서 sizeof(x)를 출력하면 4 바이트(32 비트 시스템) 또는 8 바이트(64 비트 시스템)가 출력된다.

이는 sizeof(x)가 동적으로 할당된 메모리의 크기가 아니라 포인터 변수 x 자체의 크기를 반환하기 때문이다.

반면 다음과 같은 경우에는 기대한 대로 동작한다.

```
int x[10];  
printf("%dWn", sizeof(x));
```

이 경우 컴파일러는 x가 10 개의 정수를 저장하는 배열이라는 사실을 알고 있으므로, 전체 크기인 40 바이트를 계산하여 반환할 수 있다.

문자열 할당 시 주의사항

문자열을 위한 공간을 할당할 때는 다음과 같이 작성한다.

```
malloc(strlen(s) + 1);
```

여기서 strlen() 함수로 문자열의 길이를 구한 뒤, 문자열의 끝을 나타내는 널 문자('W0')를 저장하기 위해 1 바이트를 추가한다.

이 경우 sizeof()를 사용하면 원하는 결과가 나오지 않을 수 있으므로 주의해야 한다.

malloc()의 반환 타입

malloc()은 void * 타입으로 포인터를 반환한다.

이는 해당 메모리에 어떤 타입의 데이터를 저장할지 프로그래머가 결정하도록 하기 위한 C 언어의 특징이다.

따라서 프로그래머는 타입 캐스팅(type casting)을 통해 원하는 포인터 타입으로 변환하여 사용한다.

예를 들어,

```
double *d = (double *)malloc(sizeof(double));
```

여기서 캐스팅은 실제 메모리 동작을 바꾸는 것이 아니라, 컴파일러와 다른 프로그래머에게 데이터 타입을 명시적으로 알려주는 역할을 한다.

프로그램이 동작하는 데 있어 캐스팅이 항상 필수적인 것은 아니지만, 코드의 명확성을 위해 사용되는 경우가 많다.

[3] free() 함수

메모리 관리에서 메모리 할당은 비교적 쉬운 문제에 속한다.

반면, 할당된 메모리를 언제 해제할 것인지, 그리고 정확하게 해제되었는지 확인하는 것은 훨씬 어려운 문제이다.

힙(heap) 메모리 중 더 이상 사용하지 않는 영역을 해제하기 위해 프로그래머는 free() 함수를 호출한다.

예를 들어 다음과 같은 코드가 있다고 하자.

```
int *x = malloc(10 * sizeof(int));  
...  
free(x);
```

free() 함수는 하나의 인자를 받으며, 이 인자는 malloc()에 반환된 포인터이다.

중요한 점은 free()를 호출할 때 할당된 메모리의 크기를 전달하지 않는다는 것이다.

할당된 메모리의 크기에 대한 정보는 메모리 할당 라이브러리 내부에서 관리되기 때문에, free()는 단순히 해당 포인터만 전달받아 메모리를 해제한다.

[4] 흔한 오류

malloc()과 free()를 사용하는 과정에서 다양한 오류가 자주 발생한다.

운영체제를 가르치면서 반복적으로 관찰된 대표적인 오류들을 살펴보자.

C 프로그램은 컴파일러가 성공했다고 해서 반드시 올바르게 동작하는 것은 아니다.

즉, 컴파일 성공은 올바른 프로그램의 필요조건이지만 충분조건은 아니다.

최근 많은 프로그래밍 언어들은 자동 메모리 관리(automatic memory management) 기능을 제공한다.

이러한 언어에서는 메모리를 할당하기 위해 new 와 같은 함수를 사용하지만, 메모리 해제하는 작업은 프로그래머가 직접 수행하지 않는다.

대신 쓰레기 수집기(garbage collector)가 실행되어 더 이상 참조되지 않는 메모리를 자동으로 찾아 해제한다.

하지만 C에서는 메모리를 직접 관리해야 하기 때문에 다음과 같은 오류들이 자주 발생한다.

[4-1] 메모리 할당을 잊어버리는 경우

일부 함수는 필요한 메모리가 이미 할당되어 있다고 가정한다.

예를 들어, strcpy(dst, src)는 문자열을 복사하지만, 목적지 버퍼의 메모리를 자동으로 할당하지 않는다.

잘못된 코드:

```
char *src = "hello";  
char *dst; //메모리 할당 없음  
strcpy(dst, src); //segmentation fault 발생 가능
```

이 경우 dst 는 메모리를 가리키지 않기 때문에 세그멘테이션 폴트가 발생할 가능성이 높다.

올바른 코드:

```
char *src = "hello";  
char *dst = (char *)malloc(strlen(src) + 1);  
strcpy(dst, src);
```

또는 strdup() 함수를 사용하면 더 간단하게 처리할 수 있다.

[4-2] 메모리를 부족하게 할당하는 경우(버퍼 오버플로우)

버퍼 크기를 충분히 할당하지 않으면 버퍼 오버플로우(buffer overflow)가 발생할 수 있다.

잘못된 코드:

```
char *src = "hello";  
char *dst = (char *)mallco(strlen(src)); // 공간 부족  
strcpy(dst, src);
```

문자열에는 널 종료 문자(W0)가 필요하므로 최소 strlen(src) + 1 바이트가 필요하다.

버퍼 오버플로우는 경우에 따라 다음과 같은 결과를 초래한다.

- 프로그램이 정상적으로 동작하는 것처럼 보임
- 다른 변수의 데이터를 덮어씀
- 프로그램 크래시 발생
- 보안 취약점 발생

프로그램이 한 번 정상적으로 실행되었다고 해서 프로그램이 올바르다는 의미는 아니다.

[4-3] 할당받은 메모리를 초기화하지 않는 경우

malloc()으로 메모리를 할당했지만 초기화하지 않으면 초기화되지 않은 값(uninitialized read)을 읽게된다.

이 경우 프로그램은 힙에 남아 있는 임의의 값을 읽게 된다.

가능한 결과

- 운 좋게 정상 동작
- 임의 값 사용
- 프로그램 오류 발생

따라서 할당된 메모리는 반드시 명시적으로 초기화하는 것이 좋다.

[4-4] 메모리를 해제하지 않는 경우(Memory Leak)

메모리 해제를 하지 않으면 메모리 누수가 발생한다.

이 문제는 특히 다음과 같은 프로그램에서 심각하다.

- 장시간 실행되는 프로그램
- 서버 프로그램
- 운영체제

메모리 누수가 계속 발생하면 결국 메모리가 부족해져 시스템이 중단될 수 있다.

짧은 프로그램에서는 프로세스 종료 시 운영체제가 메모리를 정리하기 때문에 문제가 없어 보일 수도 있다.

하지만 이러한 습관은 바람직하지 않다.

한 바이트라도 명시적으로 할당했다면 반드시 해제한다.

[4-5] 메모리 사용이 끝나기 전에 해제하는 경우(Dangling Pointer)

메모리를 해제했지만 그 포인터를 계속 사용하는 경우 dangling pointer 문제가 발생한다.

예를 들어,

- free() 호출 후 포인터 사용
- 해제된 메모리를 다시 malloc()이 재사용

이 경우 프로그램이

- 크래시 발생
- 다른 메모리 영역 덮어쓰기

등의 문제가 발생할 수 있다.

[4-6] 메모리를 여러 번 해제하는 경우(Double Free)

같은 메모리를 두 번 free() 하면 double free 오류가 발생한다.

이 경우

- 프로그램 크래시
- 메모리 할당 라이브러리 오류
- 예측 불가능한 동작

이 발생할 수 있다.

[4-7] 잘못된 포인터를 free()하는 경우(Invalid Free)

free() 함수는 반드시 malloc()이 반환한 포인터만 전달해야 한다.

다음과 같은 경우 문제가 발생한다.

- 스택 변수 해제
- 이미 해제된 포인터
- 잘못된 주소

이러한 invalid free 는 매우 위험한 오류이다.

[4-8] 요약

메모리 관리에서는 다양한 오류가 발생할 수 있다.

이러한 문제를 찾기 위해 여러 도구들이 존재한다.

대표적인 도구:

- Purify
- Valgrind

이 도구들은 메모리 누스, 잘못된 접근, 해제 오류 등을 찾아내는 데 매우 유용하다.

이러한 도구에 익숙해지면, 이전에는 어떻게 디버깅했는지 의문이 들 정도로 큰 도움을 받을 수 있다.

여담: 프로세스가 종료하면 왜 메모리 누수가 일어나지 않는가

실행 시간이 짧은 프로그램에서는 malloc()을 이용해 메모리를 할당한 뒤 프로그램이 곧 종료되는 경우가 있다.

이대 종료 직전에 free()를 반드시 호출해야 할까?

겉보기에는 호출하지 않으면 잘못된 것처럼 보이지만, 실제로는 큰 문제가 발생하지 않는 경우가 많다.

그 이유는 시스템이 두 단계로 메모리를 관리하기 때문이다.

1 단계: 운영체제의 메모리 관리

첫 번째 단계는 운영체제(OS)가 수행한다.

- 프로세스가 실행될 때 운영체제는 프로세스에게 메모리를 할당한다.
- 프로세스가 종료되거나 강제로 종료되면 운영체제는 그 프로세스에 할당된 모든 메모리를 회수한다.

즉, 프로세스가 죽는 순간 해당 프로세스의 주소 공간 전체가 정리된다.

2 단계: 프로세스 내부의 메모리 관리

두 번째 단계는 프로세스 내부에서 실행되는 메모리 관리이다.

이 단계에서는 다음과 같은 함수들이 사용된다.

- malloc(): 힙에서 메모리 할당
- free(): 힙에서 메모리 해제

이 함수들은 프로세스 내부 힙(heap)에서 메모리를 관리한다.

프로세스 종료 시 메모리 정리

만약 프로그래머가 free()를 호출하지 않아 메모리 누수가 발생하더라도, 프로세스가 종료될 때 운영체제가 모든 메모리를 회수한다.

이때 회수되는 메모리는 다음을 포함한다.

- 코드 영역
- 스택(stack)
- 힙(heap)
- 프로세스 주소 공간의 모든 페이지

따라서 프로세스 내부 힙의 상태와 관계없이, 프로세스가 종료되면 모든 메모리는 운영체제가 자동으로 정리한다.

즉, free()를 호출하지 않았더라도 실제 메모리 누수가 시스템 전체에 남지는 않는다.

실행 시간이 짧은 프로그램의 경우

실행 시간이 짧은 프로그램에서는 free()를 호출하지 않아도 대부분 실질적인 문제가 발생하지 않는다.

하지만 이는 좋은 프로그래밍 습관은 아니다.

장시간 실행되는 프로그램의 경우

다음과 같은 장시간 실행되는 프로그램에서는 메모리 누수가 매우 심각한 문제가 된다.

- 웹 서버
- 데이터베이스 관리 시스템(DBMS)
- 장시간 실행되는 서비스 프로그램

이러한 프로그램에서는 메모리 누수가 계속 누적되면 결국 메모리를 모두 소모하여 프로그램이 크래시하게 된다.

운영체제에서의 메모리 누수

메모리 누수는 특히 운영체제 자체에서 매우 심각한 문제이다.

운영체제는 종료되지 않는 프로그램이기 때문에, 커널 코드에서 메모리 누수가 발생하면 시스템 전체에 영향을 줄 수 있다.

이러한 이유로 커널 개발은 매우 어렵고 까다로운 작업이라고 할 수 있다.

팁: 컴파일되었다 또는 실행되었다 ≠ 원하는 결과다

프로그램이 컴파일되었다거나, 심지어 한 번 또는 여러 번 정상적으로 실행되었다고 해서 그 프로그램이 반드시 정확하다는 의미는 아니다.

프로그램이 정상적으로 동작하는 것처럼 보이는 이유는 여러 우연한 상황들이 맞아떨어졌기 때문일 수 있다.

하지만 환경이나 조건이 조금만 바뀌어도 프로그램은 갑자기 제대로 동작하지 않을 수 있다.

많은 학생들은 프로그램이 갑자기 동작하지 않으면 다음과 같은 반응을 보인다.

- “그런데 전에는 동작했어요!”
- 컴파일러를 탓한다.
- 운영체제를 탓한다.
- 하드웨어를 탓한다.
- 심지어는 교수까지 탓한다.

하지만 실제 문제는 대부분 바로 그곳, 즉 자신의 코드 안에 존재한다.

따라서 다른 요소를 탓하기 전에 해야 할 일은 다음과 같다.

자리로 돌아가서 자신의 코드를 디버깅하는 것이다.

[5] 운영체제의 자원

지금까지 malloc()과 free()를 설명하는 시스템 콜(system call)에 대해서는 거의 언급하지 않았다.

그 이유는 간단하다.

malloc()과 free()는 시스템 콜이 아니라 라이브러리 함수이기 때문이다.

malloc() 라이브러리는 프로세스의 가상 주소 공간 내부에서 메모리를 관리한다.

하지만 이 라이브러리도 결국 운영체제로부터 더 많은 메모리를 요청하거나 반환해야 한다. 이때 시스템콜을 사용한다.

[5-1] brk 시스템 콜

대표적인 시스템 콜 중 하나가 brk 이다.

brk 는 프로그램의 break 위치를 변경하는 시스템 콜이다.

여기서 break 란 힙(heap)의 마지막 위치를 의미한다.

brk 는 새로운 break 주소를 인자로 받는다.

- 새로운 break 가 현재 break 보다 크면: 힙 크기 증가
- 새로운 break 가 현재 break 보다 작으면: 힙 크기 감소

즉, brk 는 힙의 크기를 조절하는 역할을 한다.

[5-2] sbrk 시스템 콜

sbrk 는 brk 와 비슷한 역할을 한다.

차이점은 다음과 같다.

- brk: 새로운 절대 주소를 전달
- sbrk: 증가할 크기(increment)를 전달

즉, sbrk 는 힙의 크기를 얼마만큼 늘릴지를 지정하는 방식이다.

주의사항

프로그래머가 직접 brk 나 sbrk 를 호출해서는 안 된다.

이 시스템 콜들은 메모리 할당 라이브러리(malloc 라이브러리) 내부에서 사용된다.

사용자가 직접 호출하면 메모리 관리가 꼬여 심각한 문제가 발생할 수 있다.

따라서 반드시 malloc()과 free()를 사용해야 한다.

[5-3] mmap 을 이용한 메모리 할당

운영체제로부터 메모리를 얻는 또 다른 방법으로 mmap() 함수가 있다.

적절한 인자를 전달하면 mmap()은 프로그램에 anonymous 메모리 영역을 생성한다.

anonymous 메모리 영역이란 다음과 같은 특징을 가진다.

- 특정 파일과 연결되지 않은 메모리
- 스왑 공간(swap space)과 연결된 메모리

이 메모리는 힙과 유사하게 취급되고 관리된다.

mmap()에 대한 자세한 내용은 가상 메모리 부분에서 다시 다루며, 자세한 사용법은 mmap()의 매뉴얼 페이지를 참고하면 된다.

[6] 기타 함수들

메모리 할당과 관련하여 malloc()과 free() 외에도 몇 가지 유용한 함수들이 있다.

[6-1] calloc()

calloc()은 메모리를 할당한 후 해당 메모리 영역을 모두 0 으로 초기화하여 반환하는 함수이다.

즉,

- 메모리 할당
- 메모리 초기화(0 으로 설정)

이 두 작업을 동시에 수행한다.

이 함수는 메모리를 할당한 뒤 초기화를 잊어버리는 오류를 방지하는 데 도움이 된다. 특히 앞에서 설명한 초기화되지 않은 읽기 문제를 예방할 수 있다.

[6-2] realloc()

realloc()은 이미 할당된 메모리 공간의 크기를 변경할 때 사용하는 함수이다.

예를 들어, 동적으로 할당한 배열의 크기를 더 늘려야할 때 유용하다.

realloc()의 동작 방식은 다음과 같다.

1. 더 큰 새로운 메모리 영역을 확보한다.
2. 기존 메모리 영역의 데이터를 새로운 영역으로 복사한다.
3. 새로운 메모리 영역에 대한 포인터를 반환한다.

따라서 realloc()을 사용하면 기존 데이터를 유지하면서 메모리 크기를 확장할 수 있다.

[1-2] 가상화: 주소 변환의 원리

핵심 질문: 어떻게 효율적이고 유연하게 메모리를 가상화하는가

어떻게 효율적인 메모리 가상화를 구축할 수 있을까?

프로그램이 필요로 하는 유연성을 어떻게 제공하는가?

프로그램이 접근할 수 있는 메모리의 위치에 대한 제어를 어떻게 유지하는가?

메모리 접근을 어떻게 적절히 제한할 수 있을까?

어떻게 이 모든 것을 효율적으로 할 수 있을까?

CPU 가상화에서는 제한적 직접 실행(LDE:Limited Direct Execution)이라는 기법을 사용한다.

LDE의 핵심 아이디어는 매우 단순하다.

- 대부분의 경우 프로그램은 하드웨어에서 직접 실행된다.
- 하지만 특정 상황에서는 운영체제가 개입한다.

운영체제가 개입하는 대표적인 경우는 다음과 같다.

- 시스템 콜(system call)이 발생했을 때
- 타이머 인터럽트(timer interrupt)가 발생했을 때

이러한 순간에 운영체제가 하드웨어를 제어하여 문제가 발생하지 않도록 관리한다.

운영체제는 하드웨어의 약간의 지원을 이용하여 프로그램 실행을 방해하지 않으면서 가상화를 제공한다.

즉, 중요한 순간에만 개입하여 시스템을 제어한다.

결국 현대 운영체제의 목표는 다음 두 가지이다.

- 효율성: 프로그램을 최대한 빠르게 실행
- 제어: 시스템 자원을 안전하게 관리

메모리 가상화의 목표

메모리 가상화에서도 CPU 가상화와 비슷한 전략을 사용한다.

즉, 다음 두 가지를 동시에 달성하려 한다.

- 효율성
- 제어

효율성을 높이기 위해 운영체제는 하드웨어 자원을 활용한다.

처음에는 단순한 레지스터를 사용하고, 이후 더 발전된 구조인

- TLB
- 페이지 테이블

등과 같은 복잡한 하드웨어를 활용하게 된다.

제어의 의미는 다음과 같다.

- 응용 프로그램이 자신의 메모리 영역 외의 다른 메모리에 접근하지 못하도록 보장하는 것

이를 통해 다음과 같은 보호가 이루어진다.

- 프로그램 간 보호(프로그램 <-> 프로그램)
- 운영체제 보호(프로그램 <-> 운영체제)

이러한 보호를 위해서라도 하드웨어의 자원이 필요하다.

유연성(Flexibility)

메모리 가상화 시스템은 유연성도 제공해야 한다.

- 프로그래머가 원하는 방식으로 주소 공간을 사용할 수 있어야 하고
- 프로그래밍하기 쉬운 환경을 제공해야 한다.

주소 변환(Address Translation)

이 장에서 다루는 핵심 기술은 하드웨어 기반 주소 변환(hardware-based address translation)이다.

이를 간단히 주소 변환(address translation)이라고도 부른다.

이 기술은 제한적 직접 실행(LDE) 방식에 추가적으로 사용되는 기능이라고 볼 수 있다.

주소 변환의 역할은 다음과 같다.

프로그램이 사용하는 가상 주소(virtual address)를 실제 메모리 위치인 물리 주소(physical address)로 변환하는 것이다.

- 명령어 반입
- 데이터 로드(load)
- 데이터 저장(store)

과 같은 모든 메모리 접근에서 가상 주소 → 물리 주소 변환이 이루어진다.

이 변환 과정은 하드웨어가 수행한다.

운영체제의 역할

하드웨어만으로는 메모리 가상화를 완전히 구현할 수 없다.

하드웨어가 올바르게 동작하도록 하기 위해 운영체제가 반드시 관여해야 한다.

운영체제는 다음과 같은 역할을 수행한다.

- 메모리의 사용 중인 영역과 빈 영역 관리
- 메모리 할당 및 보호
- 주소 변환을 위한 하드웨어 설정

즉, 운영체제는 메모리 사용을 전체적으로 관리하고 제어한다.

메모리 가상화의 궁극적인 목표

메모리 가상화의 궁극적인 목표는 다음과 같다.

- 각 프로그램이 자신만의 전용 메모리를 가지고 있는 것처럼 보이게 만드는 것

즉, 프로그램은 다음과 같은 환상을 가지게 된다.

- 자신의 코드와 데이터가 자기만의 독립된 메모리 공간에 존재한다고 느낀다.

하지만 실제로는 다음과 같은 현실이 존재한다.

- 많은 프로그램이 같은 물리 메모리를 공유한다.
- CPU가 프로그램 사이를 전환하면서 실행된다.

운영체제는 하드웨어의 도움을 받아 이러한 복잡한 현실을 숨기고, 시스템을 유용하고 강력하며 사용하기 쉬운 환경으로 만들어 준다.

[1] 가정

메모리 가상화를 설명하기 위한 첫 번째 접근 방법은 매우 단순하다.
심지어 약간 비현실적으로 보일 정도로 단순한 가정을 사용한다.

하지만 이러한 단순한 모델을 먼저 이해하는 것이 중요하다.
왜냐하면 이후에는 TLB, 멀티 레벨 페이지 테이블 등 훨씬 복잡한 개념을 이해해야 하기 때문이다.

[1-1] 기본 가정

메모리 가상화를 설명하기 위해 다음과 같은 가정을 둔다.

1. 주소 공간은 물리 메모리에 연속적으로 배치된다.

- 각 프로세스의 주소 공간(address space)은 물리 메모리 안에서 연속적인 영역에 배치된다고 가정한다.
- 즉, 프로그램의 메모리가 물리 메모리에서 끊어지지 않고 하나의 블록으로 존재한다고 본다.

2. 주소 공간의 크기는 크지 않다.

- 프로세스의 주소 공간 크기는 물리 메모리보다 작다고 가정한다.
- 즉, 프로세스 주소 공간 < 물리 메모리 크기
- 이 가정 덕분에 메모리 관리가 단순해진다.

3. 모든 프로세스의 주소 공간 크기는 동일하다.

- 모든 프로그램이 같은 크기의 주소 공간을 가진다고 가정한다.
- 이 또한 설명을 단순하게 만들기 위한 가정이다.

[1-2] 가정의 의미

이러한 가정들이 실제 시스템과는 다소 비현실적일 수 있다.

하지만 이 가정들은 메모리 가상화의 기본 개념을 설명하기 위한 출발점이다.

이후 설명이 진행되면서 이러한 가정들은 점차 완화되고, 결국 실제 운영체제에서 사용하는 현실적인 메모리 가상화 기법으로 발전하게 된다.

[2] 사례

주소 변환(address translation)이 왜 필요한지 이해하기 위해 간단한 예제를 살펴보자.
예를 들어, 다음과 같은 C 코드가 있다고 가정하자.

```
void func(){  
    int x= 3000;  
    x = x + 3;  
}
```

이 코드는 변수 x의 값을 3만큼 증가시키는 간단한 연산을 수행한다.

[2-1] 컴파일 후 어셈블리 코드

컴파일러는 위 코드를 다음과 같은 x86 어셈블리 코드로 변환할 수 있다.

```
128: movl 0x0(%ebx), %eax    ; ebx가 가리키는 메모리 값을 eax로 로드  
132: addl $0x03, %eax        ; eax에 3을 더함  
135: movl %eax, 0x0(%ebx)    ; eax 값을 다시 메모리에 저장
```

각 명령어의 의미는 다음과 같다.

1. movl
- ebx가 가리키는 메모리 값을 eax 레지스터로 가져온다.
2. addl
- eax 레지스터 값에 3을 더한다.
3. movl
- 계산된 값을 다시 같은 메모리 위치에 저장한다.

여기서 변수 x의 주소는 레지스터 ebx에 저장되어 있다고 가정한다.

[2-2] 프로세스 주소 공간에서의 배치

프로세스의 주소 공간은 다음과 같이 구성되어 있다고 가정한다.

- 코드 영역: 주소 128
- 변수 x: 주소 15KB
- x의 초기값: 3000

즉, 코드와 데이터가 프로세스의 가상 주소 공간에 배치되어 있다.

[2-3] 프로그램 실행 시 발생하는 메모리 접근

이 프로그램이 실행되면 다음과 같은 메모리 접근이 발생한다.

1. 주소 128의 명령어를 가져옴
2. 해당 명령어 실행 → 주소 15KB에서 데이터 로드
3. 주소 132의 명령어를 가져옴
4. 해당 명령어 실행 → 레지스터 연산(메모리 접근 없음)
5. 주소 135의 명령어를 가져옴
6. 해당 명령어 실행 → 15KB 위치에 결과 저장

[2-4] 프로그램이 보는 주소 공간

프로그램의 관점에서는 주소 공간이 다음과 같이 보인다.

0 ~ 16KB

즉, 프로그램은 주소 0 부터 시작하는 연속된 메모리 공간을 가진다고 생각한다.
그리고 프로그램이 사용하는 모든 주소는 이 범위 안에 있어야 한다.

[2-5] 실제 물리 메모리에서의 배치

하지만 운영체제는 프로세스를 항상 물리 주소 0 에서 실행시키지 않는다.

예를 들어, 물리 메모리가 다음과 같이 구성될 수 있다.

0KB ~ 16KB	→ 운영체제 사용
16KB ~ 32KB	→ 빈 공간
32KB ~ 48KB	→ 현재 프로세스
48KB ~ 64KB	→ 빈 공간

- 프로그램은 주소 0 부터 시작한다고 생각하지만
- 실제로는 물리 주소 32KB 위치에서 실행되고 있다.

[3] 동적(하드웨어 기반)재배치

하드웨어 기반 주소 변환을 이해하기 위해 가장 먼저 등장한 기법이 베이스와 바운드(Base and Bound) 방식이다.

이 방식은 동적 재배치(Dynamic Relocation)라고도 부른다.

두 용어는 같은 의미로 사용된다.

이 기법은 1950년대 후반 시분할 컴퓨터에서 처음 사용되었다.

[3-1] 필요한 하드웨어

이 기법을 통해 CPU에는 두 개의 특별한 레지스터가 필요하다.

- Base Register(베이스 레지스터)
- Bound Register(바운드/Limit 레지스터)

이 두 레지스터를 이용하면

1. 프로세스를 원하는 물리 메모리 위치에 배치할 수 있고
2. 프로세스가 자기 메모리의 영역만 접근하도록 보호할 수 있다.

[3-2] 기본 동작 원리

프로그램은 항상 주소 0부터 시작하는 것처럼 작성되고 컴파일된다.

하지만 실제 실행 시에는 운영체제가 프로그램을 다른 물리 메모리 위치에 배치할 수 있다.

예를 들어,

- 운영체제가 프로그램을 물리 주소 32KB 위치에 배치했다고 가정한다.
- 그러면 Base Register = 32KB로 설정된다.

[3-3] 주소 변환 방식

프로그램이 생성하는 주소는 가상 주소(virtual address)이다.

CPU는 다음 공식을 이용하여 물리 주소(physical address)를 만든다.

$$\text{Physical Address} = \text{Virtual Address} + \text{Base}$$

즉, 가상 주소 + 베이스 값 = 실제 물리 주소

[3-4] 실행 과정 예시

다음과 같은 명령어가 있다고 하자.

```
128: movl 0x0(%EBX), %eax
```

프로그램 카운터(PC)는 128 이다.

하지만 실제 실행 과정에서는 다음과 같이 변환된다.

명령어 반입

가상 주소 = 128

Base = 32768 (32KB)

물리 주소 = 128 + 32768
= 32896

데이터 접근

프로그램이 가상 주소 15KB 의 데이터를 읽으려고 한다면

물리 주소 = 15KB + 32KB
= 47KB

CPU 는 물리 주소 47KB 에서 데이터를 가져온다.

[3-5] 왜 “동적 재배치”인가

이 주소 변환은 프로그램 실행 중에 동적으로 발생한다.

즉,

- 프로그램은 항상 0 부터 시작하는 주소 공간을 사용한다고 생각하지만
- 실제로는 다른 물리 위치에서 실행된다.

또한 실행 중에도 주소 공간을 다른 위치로 이동할 수 있다.

그래서 이를 동적 재배치라고 부른다.

[3-6] Bound 레지스터의 역할(메모리 보호)

지금까지 Base 레지스터만 사용한 것처럼 보인다.

그렇다면 Bound 레지스터는 무엇을 할까?

Bound 레지스터의 역할은 메모리 보호이다.

CPU 는 메모리 접근 전에 다음을 확인한다.

Virtual Address < Bound

즉, 가상 주소가 프로세스의 주소 공간 범위 안에 있는지 검사한다.

예를 들어,

Bound = 16KB

- 0~16KB: 접근 가능
- 16KB 이상: 예외 발생

범위를 벗어나면

- CPU 가 예외를 발생
- 프로세스 종료

[3-7] MMU(Memory Management Unit)

Base 와 Bound 레지스터는 CPU 내부 하드웨어 구조이다.

주소 변환을 담당하는 CPU 의 이 부분을 MMU(Memory Management Unit)라고 부른다.

메모리 관리 기술이 발전하면서 MMU 에는

- 페이지 테이블
- TLB

같은 더 복잡한 하드웨어 기능들이 추가된다.

[3-8] Bound 레지스터 구현 방식

Bound 레지스터는 두 가지 방식으로 구현할 수 있다.

방식 1: 일반적인 방식

Bound 레지스터에 주소 공간 크기를 저장한다.

Virtual Address < Bound

확인 후

Physical Address = Base + Virtual Address

방식 2

Bound 레지스터에 마지막 물리 주소를 저장한다.

Physical Address = Base + Virtual Address

계산 후

Physical Address < Bound

두 방식은 논리적으로 동일하다.

[3-9] 예제

주소 공간 크기: 4KB

프로세스 시작 위치: 16KB

Base = 16KB

Bound = 4KB

가상 주소	물리 주소
0	16KB
1KB	17KB
3000	19384
4400	Fault (범위 초과)

팁: 하드웨어 기반의 동적 재배치

동적 재배치를 구현하기 위해서는 작은 하드웨어면 큰 성공을 거둔다.

즉, 베이스 레지스터는 프로그램에 의해 생성된 가상 주소를 물리 주소로 변환하는 데 사용된다.

베이스(또는 한계) 레지스터는 이 가상 주소가 주소 공간의 범위 내에 있다는 것을 보장한다.

두 레지스터는 함께 간단하고 효율적인 메모리의 가상화를 제공한다.

[4] 하드웨어 지원: 요약

동적 재배치(Base & Bound)를 구현하려면, 운영체제만으로는 부족하고 하드웨어의 지원이 꼭 필요하다.

이 절에서는 그 하드웨어 요구사항을 정리한다.

[4-1] 두 가지 CPU 모드가 필요하다.

CPU에는 두 가지 실행 모드가 있어야 한다.

- 커널 모드(특권 모드): 운영체제가 실행되는 모드
- 사용자 모드: 응용 프로그램이 실행되는 모드

운영체제는 시스템 전체 자원에 접근할 수 있어야 하므로 커널 모드에서 실행된다.

반면, 사용자 프로그램은 위험한 작업을 마음대로 수행하면 안 되므로 사용자 모드에서 실행되며, 할 수 있는 일이 제한된다.

현재 CPU가 어떤 모드에 있는지 PSW(Processor Status Word)의 한 비트가 나타낸다.

그리고 다음과 같은 상황에서는 CPU 모드가 전환된다.

- 시스템 콜 발생
- 예외 발생
- 인터럽트 발생

즉, 평소에도 사용자 프로그램이 실행되다가, 중요한 사건이 생기면 CPU가 운영체제로 제어를 넘긴다.

[4-2] Base/Bound 레지스터가 필요하다.

하드웨어는 각 CPU에 대해 Base 레지스터와 Bound 레지스터를 제공해야 한다.

이 두 레지스터는 MMU(Memory Management Unit)의 일부로 동작한다.

- Base 레지스터: 프로세스가 실제로 올라가 있는 물리 메모리의 시작 주소
- Bound 레지스터: 프로세스가 접근 가능한 주소 공간의 크기 또는 범위

프로그램이 실행되면서 가상 주소를 만들면, 하드웨어는 이를 다음과 같이 변환한다.

물리 주소 = 가상 주소 + Base

이와 동시에, 그 가상 주소가 프로세스에게 허용된 범위 안에 있는지 Bound 레지스터로 검사한다.

즉, 하드웨어는 다음 두 가지를 동시에 해야 한다.

- 주소 변환
- 범위 검사

[4-3] 주소 변환과 범위 검사를 위한 회로가 필요하다

단순히 레지스터만 있다고 끝나는 것이 아니다.

CPU 내부에는 실제로 다음 작업을 수행하는 하드웨어 회로가 있어야 한다.

- 가상 주소에 Base 값을 더하는 회로
- 해당 주소가 Bound 범위 안에 있는지 검사하는 회로

동적 재배치에서의 주소 변환 자체는 비교적 단순하다.

하지만 이 단순한 기능도 반드시 하드웨어 수준에서 빠르게 처리되어야 한다.

[4-4] Base/Bound 값을 바꾸는 특권 명령어가 필요하다.

운영체제가 프로세스를 바꿔 실행할 때는, 새 프로세스에 맞게 Base/Bound 레지스터 값도 함께 변경해야 한다.

이를 위해 하드웨어는 Base/Bound 값을 설정하는 명령어를 제공해야 한다.

그런데 이 명령어는 아무나 실행하면 안 된다.

만약 사용자 프로그램이 자기 마음대로 Base 값을 바꿀 수 있다면, 다른 프로세스의 메모리나 운영체제 메모리에 접근하려고 시도할 수 있기 때문이다.

따라서 이 명령어들은 반드시 특권 명령어(privileged instruction)여야 하며, 커널 모드에서만 실행 가능해야 한다.

[4-5] 예외 핸들러 등록 기능이 필요하다

CPU가 예외를 발생시켰을 때, 어디로 제어를 넘겨야 하는지도 알아야 한다.

그래서 운영체제는 하드웨어에게 예외 처리 코드(예외 핸들러)의 위치를 알려줄 수 있어야 한다.

이를 위해 하드웨어는 예외 핸들러를 등록하는 특권 명령어를 제공해야 한다.

즉, 운영체제는 미리 다음과 같은 상황에 대한 처리 루틴을 등록해 둔다.

- 범위를 벗어난 메모리 접근
- 사용자 모드에서 특권 명령어 실행
- 기타 예외 상황

[4-6] 예외 발생 기능이 필요하다

하드웨어는 잘못된 동작이 발생했을 때 예외(exception)를 일으킬 수 있어야 한다.
대표적인 경우는 다음과 같다.

(1) 범위를 벗어난 메모리 접근

사용자 프로그램이 자신의 Bound 범위를 넘어서는 주소를 접근하려 하면, CPU 는 예외를 발생시켜야 한다.

그러면 현재 프로그램 실행은 중단되고, 운영체제의 “범위 벗어남” 예외 핸들러가 실행된다.
대부분의 경우 운영체제는 이런 프로세스를 강제 종료한다.

(2) 사용자 모드에서 특권 연산 실행

사용자 프로그램이 Base/Bound 레지스터를 변경하는 등의 특권 명령어를 실행하려 해도 예외가 발생해야 한다.

이 경우 CPU 는 “사용자 모드의 특권 연산” 예외 핸들러로 제어를 넘긴다.

[5] 운영체제 이슈

동적 재배치를 지원하기 위해서는 하드웨어 기능만으로는 충분하지 않고, 운영체제의 관리도 함께 필요하다.

베이스와 바운드 방식을 이용한 가상 메모리에서는 운영체제가 여러 시점에서 개입해야 한다.

첫째, 프로세스가 생성될 때 운영체제는 프로세스의 주소 공간을 저장할 물리 메모리 공간을 찾아 할당해야 한다.

본 장의 가정처럼 각 주소 공간의 크기가 일정하고 물리 메모리보다 작다면, 운영체제는 물리 메모리를 일정한 크기의 슬롯들로 보고 관리할 수 있다.

이때 빈 공간 리스트를 이용해 사용 가능한 슬롯을 찾고, 선택된 공간을 사용 중으로 표시한다.

둘째, 프로세스가 종료할 때 운영체제는 해당 프로세스가 사용하던 메모리를 회수해야 한다. 정상 종료이든 비정상 종료이든, 종료된 프로세스의 메모리는 다시 빈 공간 리스트에 반환되고, 관련 자료구조도 함께 정리된다.

셋째, 문맥 교환이 일어날 때 운영체제는 베이스와 바운드 레지스터 값을 저장하고 복원해야 한다.

CPU에는 한 쌍의 베이스-바운드 레지스터만 존재하므로, 프로세스가 바뀔 때마다 현재 프로세스의 값을 PCB에 저장하고 다음 프로세스의 값을 CPU에 다시 설정해야 한다.

또한 프로세스가 중단된 상태에서는 주소 공간을 다른 물리 위치로 옮길 수 있으며, 이 경우 운영체제는 메모리를 새 위치로 복사한 뒤 PCB에 저장된 베이스 값을 갱신한다.

프로세스는 다시 실행될 때 자신이 다른 위치로 옮겨졌다는 사실을 알지 못한다.

마지막으로 운영체제는 예외 처리기를 준비해야 한다.

운영체제는 부팅 시 특권 명령어를 사용해 예외 핸들러를 설치하며, 프로세스가 바운드를 벗어나 메모리에 접근하는 등 불법적인 동작을 하면 CPU가 예외를 발생시키고 운영체제가 개입한다.

일반적으로 운영체제는 이런 프로세스를 종료하여 시스템을 보호한다.

결국 이 방식에서 주소 변환 자체는 하드웨어가 수행하지만, 운영체제는 메모리 할당과 회수, 문맥 교환 시 레지스터 관리, 그리고 예외 처리까지 담당하면서 전체적인 가상 메모리 동작을 뒷받침한다.

[1-3] 가상화: 세그멘테이션

핵심 질문: 대용량 주소 공간을 어떻게 지원하는가

스택과 힙 사이에 잠재적으로 큰 빈 영역이 존재하는 주소 공간을 어떻게 지원하는가?

우리의 예에서 사용하는 작은 주소 공간에서는 낭비가 그렇게 심하지 않다.

그러나 크기가 4GB 인 32 비트 주소 공간을 상상해 보라.

통상 프로그램은 단지 수 메가바이트만 사용함에도 불구하고 주소 공간 전체가 메모리에 탑재되어야 한다.

[1] 세그멘테이션: 베이스/바운드의 일반화

세그멘테이션은 베이스와 바운드 기법을 확장한 방식이다.

기존 방식에서는 주소 공간 전체에 대해 하나의 베이스 레지스터와 하나의 바운드 레지스터만 사용했지만, 세그멘테이션에서는 주소 공간의 각 논리적 세그먼트마다 베이스와 바운드 쌍을 둔다.

여기서 세그먼트란 일정한 길이를 가지는 연속적인 주소 공간을 의미하며, 대표적으로 코드, 힙, 스택 세그먼트가 있다.

이 방식의 장점은 운영체제가 각 세그먼트를 물리 메모리의 서로 다른 위치에 독립적으로 배치할 수 있다는 점이다.

따라서 실제로 사용하지 않는 가상 주소 공간까지 물리 메모리에 할당할 필요가 없어진다. 즉, 사용 중인 메모리에만 물리 공간을 할당할 수 있어서 드문 드문 사용되는 큰 주소 공간도 효율적으로 지원할 수 있다.

세그멘테이션을 지원하기 위해 MMU 는 세그먼트마다 베이스와 바운드 레지스터를 가져야 한다.

예를 들어, 코드, 힙, 스택의 세 세그먼트가 있다면 총 세 쌍의 베이스와 바운드 레지스터가 필요하다.

각 베이스 레지스터는 해당 세그먼트가 위치한 물리 메모리 시작 주소를 나타내고, 각 바운드 레지스터는 그 세그먼트의 크기를 저장한다.

주소 변환은 먼저 어떤 가상 주소가 어느 세그먼트에 속하는지를 판단한 뒤, 그 세그먼트 내부의 오프셋을 구해서 수행한다.

예를 들어, 가상 주소 100 은 코드 세그먼트에 속하므로, 코드 세그먼트의 베이스가 32KB 라면 물리 주소는 $32KB + 100$ 이 된다. 또한 100 이 코드 세그먼트의 크기인 2KB 보다 작으므로 유효한 접근이다.

반면 가상 주소 4200 은 힙 세그먼트에 속한다.

이때 단순히 4200 을 힙의 베이스에 더하면 안 되고, 먼저 힙이 가상 주소 4096 에서 시작하므로 세그먼트 내부 오프셋을 계산해야 한다.

즉, $4200 - 4096 = 104$ 가 오프셋이 되고, 여기에 힙의 베이스 주소 34KB 를 더해 물리 주소를 얻는다.

만약 어떤 주소가 해당 세그먼트의 크기를 넘어서면 하드웨어는 이를 범위 밖 접근으로 판단하고 운영체제에 트랩을 발생시킨다.
운영체제는 보통 해당 프로세스를 종료한다.
이것이 바로 세그먼트 위반(segment violation) 또는 세그먼트 폴트(segment fault)의 의미이다.

[2] 세그먼트 종류의 파악

세그멘테이션에서는 각 세그먼트마다 별도의 베이스와 바운드 값을 사용해 주소 변환을 수행해야 한다.

CPU 나 MMU 는 어떤 가상 주소가 코드 세그먼트인지, 힙 세그먼트인지, 스택 세그먼트인지 어떻게 구분할까?

즉, 단순히 가상 주소 하나만 보고도 “이 주소가 어느 세그먼트에 속하는지”를 먼저 알아내야, 그에 맞는 베이스/바운드 레지스터를 사용할 수 있다.

[2-1] 가장 일반적인 방법: 가상 주소의 상위 비트로 세그먼트 부분

가장 흔한 방식은 가상 주소의 최상위 비트 몇 개를 세그먼트 번호로 사용하는 것이다.

즉, 주소 자체를 다음과 같이 나누는 것이다.

- 상위 비트: 어떤 세그먼트인지 표시
- 하위 비트: 그 세그먼트 내부에서의 오프셋(offset)

예를 들어, 주소 공간 안에 다음과 같은 3 개의 세그먼트가 있다고 하자.

- 코드, 힙, 스택

세그먼트가 3 개이므로, 이를 구분하려면 최소 2 비트가 필요하다.

왜냐하면 1 비트로는 2 개만 구분할 수 있고, 2 비트면 최대 4 개까지 구분할 수 있기 때문이다.

즉, 14 비트 가상 주소를 다음처럼 나눌 수 있다.

- 상위 2 비트: 세그먼트 번호
- 하위 12 비트: 세그먼트 내부 오프셋

이 구조는 다음 의미를 가진다.

- 00: 코드 세그먼트
- 01: 힙 세그먼트
- 10: 스택 세그먼트
- 11: 사용하지 않음

즉, MMU 는 가상 주소가 들어오면 먼저 맨 앞 2 비트를 보고 어떤 세그먼트에 속하는지를 판단한 뒤, 해당 세그먼트에 해당하는 베이스와 바운드 값을 선택한다.

[2-2] 오프셋이 중요한 이유

세그멘테이션에서 주소 변환은 단순히 가상 주소 전체를 베이스에 더하는 방식이 아니다.

반드시 먼저 다음을 구해야 한다.

- 해당 세그먼트 내부에서 몇 번째 바이트인가?

이 값이 바로 오프셋이다.

예를 들어 힙 세그먼트가 가상 주소 공간에서 4KB 지점부터 시작한다고 하자.

어떤 가상 주소가 4200 이라면, 이 주소는 힙 세그먼트에 속하므로 실제 힙 내부에서의 위치는 다음과 같이 계산된다.

- 힙 시작 주소: 4096

- 참조 주소: 4200

- 오프셋: $4200 - 4096 = 104$

즉, 이 주소는 힙의 시작점으로부터 104 바이트 떨어진 위치를 뜻한다.

그 다음에야 물리 메모리의 힙 세그먼트 시작 주소(베이스)에 이 오프셋 104 를 더해서 최종 물리 주소를 얻는다.

즉, 핵심은

- 물리 주소 = 해당 세그먼트의 베이스 + 세그먼트 내부 오프셋

[2-3] 예시: 가상 주소 4200 의 반환

앞 예시를 그대로 사용해 보자.

- 가상 주소 크기: 14 비트

- 상위 2 비트: 세그먼트 번호

- 하위 12 비트: 오프셋

- 힙 세그먼트의 베이스: 34KB

- 힙 세그먼트의 크기: 2KB

가상 주소 4200 을 2 진수로 표현하면, 상위 2 비트가 01 이 되어 힙 세그먼트를 뜻한다.
그리고 하위 12 비트는 힙 내부 오프셋 값이 된다.

01 0000 0110 1000

이 오프셋은 104 이고, 힙 세그먼트의 물리적 시작 주소는 $34KB = 34816$ 이므로,

- 물리 주소 = $34816 + 104 = 34920$

따라서 가상 주소 4200 은 물리 주소 34920 으로 변환된다.

여기서 중요한 점은

4200 전체를 베이스에 더하는 것이 아니라, 세그먼트 식별 후 오프셋만 추출하여 더해야 한다는 것이다.

[2-4] 바운드 검사: 왜 필요한가?

세그멘테이션에서 세그먼트는 크기가 정해져 있으므로 오프셋이 그 범위를 넘어가면 안 된다.

예를 들어, 힙의 크기가 2KB 라면, 허용 가능한 오프셋 범위는 다음과 같다.

- 최소: 0
- 최대: 2047

그런데 어떤 주소가 계산된 결과 힙 내부 오프셋이 3000 이라면, 이는 힙 세그먼트의 크기를 초과하는 것이므로 잘못된 접근이다.

따라서 MMU 는 주소 변환 전에 다음 검사를 수행해야 한다.

오프셋 < 바운드

- 참이면 정상 주소
- 거짓이면 범위를 벗어난 주소

범위를 벗어나면 하드웨어는 즉시 예외를 발생시키고, 운영체제는 이 프로세스를 종료하거나 오류를 처리한다.

[2-5] 주소 변환 과정 정리

MMU 가 세그멘테이션 방식으로 주소를 변환하는 절차는 다음과 같다.

1. 가상 주소가 들어온다.
2. 상위 비트를 읽어 어떤 세그먼트인지 판단한다.
3. 하위 비트를 읽어 그 세그먼트 내부 오프셋을 구한다.
4. 오프셋이 바운드보다 작은지 검사한다.
5. 범위 안이면 베이스 + 오프셋으로 물리주소를 계산한다.
6. 범위를 벗어나면 예외를 발생시킨다.

즉, 세그멘테이션은 단순히 주소를 옮기는 기술이 아니라, 재배치(relocation)와 보호(protection)를 동시에 수행하는 구조라고 볼 수 있다.

[2-6] 코드

```
// 14 bit VA 의 상위 2bit 를 추출
Segment = (VirtualAddress & SEG_MASK) >> SEG_SHIFT

//이제 오프셋을 추출
Offset = VirtualAddress & OFFSET_MASK
if(Offset >= Boundsp[Segment])
    RaiseException(PROTECTION_FAULT)
else
    PhysAddr = Base[Segment] + Offset
Register = AccessMemory(PhysAddr)
```

[2-7] 왜 세그먼트 하나가 남는가

세그먼트는 3 개뿐인데, 세그먼트를 나타내기 위해 2 비트를 쓰면 총 4 개를 표현할 수 있다.

- 00, 01, 10, 11

그런데 실제로는 코드, 힙, 스택의 3 개만 사용하므로 한 조합은 남게 된다.

즉, 주소 공간의 일부가 사용되지 못하는 비효율이 생긴다. (전체 주소 공간의 1/4 은 사용 불가능)

이를 줄이기 위해 어떤 시스템은

- 코드와 힙을 하나의 세그먼트로 합치고
- 스택만 따로두어 세그먼트 선택에 1 비트만 사용하기도 했다.

즉, 세그먼트 구분 방식은 하드웨어 설계와 주소 공간 활용 효율 사이에서 타협한 결과라고 볼 수 있다.

[2-8] 다른 방법: 묵시적(implicit) 세그먼트 구분

세그먼트를 항상 주소 비트로만 구분하는 것은 아니다.

어떤 시스템은 주소가 어떻게 만들어졌는지를 보고 세그먼트를 추론하기도 한다.

예를 들어,

- 프로그램 카운터(PC)에서 나온 주소
 - 명령어를 가져오기 위한 것이므로 코드 세그먼트
- 스택 포인터(SP) 또는 베이스 포인터(BP) 기반 주소
 - 함수 호출, 지역 변수 접근 등이므로 스택 세그먼트
- 그 외 일반 데이터 접근 주소
 - 보통 힙 세그먼트

이 처럼 주소의 비트 패턴이 아니라 주소 생성 맥락으로 세그먼트를 판단하는 방식이 묵시적 방식이다. 이 방식은 주소 비트를 아낄 수 있다는 장점이 있지만, 하드웨어 해석이 더 복잡해질 수 있다.

[4] 공유 자원

세그멘테이션이 도입된 이후, 시스템 설계자들은 이 기법이 단순히 주소 공간을 나누는 기능만 하는것이 아니라, 메모리 사용 효율을 높이는데에도 매우 유용하다는 사실을 깨닫게 되었다.

그 대표적인 예가 바로 세그먼트 공유(sharing)이다.

기본적으로 각 프로세스는 자신만의 독립적인 주소 공간을 가진다.

즉, 어떤 프로세스가 사용하는 코드, 데이터, 힙, 스택은 다른 프로세스와 분리되어 있다고 가정한다.

이렇게 해야 한 프로세스의 잘못된 접근이 다른 프로세스의 메모리를 침범하지 못하고, 운영체제도 각 프로세스를 안전하게 격리할 수 있다.

그런데 모든 내용을 완전히 분리해서 저장하면 같은 내용이 메모리에 여러 번 중복 저장되는 비효율이 생긴다.

특히 여러 프로세스가 동일한 프로그램 코드를 실행하는 경우가 그렇다.

예를 들어, 여러 사용자가 같은 편집기 프로그램이나 브라우저 프로그램을 실행한다고 생각해보자.

각 프로세스는 서로 다른 작업 상태와 데이터를 가지지만, 프로그램의 실행 코드 자체는 거의 동일하다. 이때 각 프로세스마다 같은 코드를 물리 메모리에 따로 복사해 두면 메모리가 크게 낭비된다.

그래서 운영체제와 하드웨어는 동일한 코드 세그먼트를 여러 프로세스가 함께 참조하도록 하는 방식을 지원하게 되었다.

이것이 바로 세그먼트 공유이다.

[4-1] 왜 공유가 필요한가

세그먼트 공유가 가장 큰 목적은 메모리 절약이다.

특히 코드 세그먼트는 다음과 같은 이유로 공유하기에 적합하다.

- 같은 프로그램을 실행하는 여러 프로세스에서 내용이 동일하다.
- 일반적으로 실행 중 코드 자체는 수정되지 않는다.
- 읽기와 실행만 가능하게 제한하면 여러 프로세스가 함께 써도 안전하다.

즉, 프로세스마다 코드 세그먼트를 별도로 둘 필요 없이, 하나의 물리 메모리 영역에 저장된 코드 세그먼트를 여러 가상 주소 공간이 공통으로 가리키게 하면 된다.

이렇게 하면 물리 메모리 사용량이 줄어들고, 프로그램 로딩도 더 효율적으로 이루어진다.

[4-2] 공유를 가능하게 하려면 무엇이 필요한가?

세그먼트를 공유하려면 단순히 베이스와 바운드 정보만으로는 부족하다.

왜냐하면 운영체제는 어떤 세그먼트가

- 읽기 가능한지
- 쓰기 가능한지
- 실행 가능한지

를 구분할 수 있어야 하기 때문이다.

이를 위해 하드웨어는 각 세그먼트마다 protection bit(보호 비트)를 추가로 둔다.

이 보호 비트는 해당 세그먼트에 허용된 접근 권한을 나타낸다.

예를 들어, 세그먼트별 권한은 다음처럼 설정할 수 있다.

- 코드 세그먼트: 읽기 가능, 실행 가능, 쓰기 불가
- 힙 세그먼트: 읽기 가능, 쓰기 가능, 실행 불가
- 스택 세그먼트: 읽기 가능, 쓰기 가능, 실행 불가

이 권한 설정은 매우 중요하다.

특히 코드 세그먼트를 읽기 전용(read-only)으로 만들면, 여러 프로세스가 같은 코드 세그먼트를 공유하더라도 어느 한 프로세스가 내용을 바꿔 다른 프로세스에 영향을 주는 일이 발생하지 않는다.

즉, 공유는 가능하되 변경은 불가능하게 만들어 안전성을 확보하는 것이 핵심이다.

[4-3] 프로세스 입장에서 왜 문제가 없어 보이는가?

흥미로운 점은, 프로세스는 자신이 실제로 공유 메모리를 쓰고 있다는 사실을 보통 알지 못한다는 것이다.

각 프로세스는 여전히 자기만의 가상 주소 공간을 가진다고 생각한다.

예를 들어, 어떤 프로세스는 자신의 코드가 가상 주소 0 번지부터 시작한다고 인식할 수 있다.

다른 프로세스도 마찬가지로 자기 코드가 자기 주소 공간 안에 있다고 생각한다.

하지만 운영체제는 뒤에서 몰래 이들 가상 주소를 동일한 물리 메모리의 코드 세그먼트에 연결해 둔다.

즉, 겉으로는 각자 독립적인 주소 공간처럼 보이지만, 실제 물리 메모리 차원에서는 같은 코드 영역을 함께 쓰는 것이다.

이것이 바로 가상 메모리의 중요한 특징 중 하나인 “독립성의 환상”이다.

프로세스는 자신만의 메모리를 사용하는 것처럼 보이지만, 운영체제는 필요에 따라 일부 영역을 효율적으로 공유하게 만들 수 있다.

[4-4] 예시

세그먼트 레지스터에 단순한 베이스와 크기 정보만 있는 것이 아니라, 추가적으로 증가 방향과 보호 정보까지 포함될 수 있음을 보여준다.

예를 들어,

- 코드 세그먼트
 - 베이스: 32KB
 - 크기: 2KB
 - 양수 방향 증가: 1
 - 보호: 읽기-실행
- 힙 세그먼트
 - 베이스: 34KB
 - 크기: 2KB
 - 양수 방향 증가: 1
 - 보호: 읽기-쓰기
- 스택 세그먼트
 - 베이스: 28KB
 - 크기: 2KB
 - 양수 방향 증가: 0
 - 보호: 읽기-쓰기

여기서 보호 정보는 단순한 부가 정보가 아니라, 하드웨어가 메모리 접근의 적법성을 판단하는 핵심 기준이 된다.

예를 들어, 코드 세그먼트는 읽기와 실행만 가능하므로, 사용자가 코드 영역에 데이터를 써 넣으려고 하면 허용되지 않는다.

반대로 힙과 스택은 읽기/쓰기가 가능하지만, 일반적으로 실행은 허용되지 않는다.

이는 악성 코드 삽입이나 잘못된 실행을 막는 데도 중요한 역할을 한다.

[4-5] 하드웨어 알고리즘은 어떻게 달라지는가?

기존의 세그멘테이션에서는 하드웨어가 두 가지를 중심으로 검사했다.

1. 해당 주소가 어떤 세그먼트에 속하는가
2. 그 오프셋이 바운드 안에 있는가

즉, 세그먼트 범위만 벗어나지 않으면 접근을 허용했다.

하지만 protection bit 가 추가되면 이것만으로는 충분하지 않다.

이제는 범위 안에 있는 주소라 하더라도, 수행하려는 동작이 허용된 권한인지까지 검사해야 한다.

즉, 하드웨어는 다음 세 단계를 확인해야 한다.

1. 주소가 어느 세그먼트에 속하는지 판별
2. 오프셋이 해당 세그먼트의 바운드 안에 있는지 검사
3. 현재 요청한 접근 방식(읽기/쓰기/실행)이 보호 비트에 의해 허용되는지 검사

이 세 조건을 모두 만족해야만 실제 메모리 접근이 가능하다.

[4-6] 어떤 경우 예외가 발생하는가?

protection bit 가 도입되면 다음과 같은 잘못된 접근을 하드웨어가 즉시 탐지할 수 있다.

(1) 읽기 전용 영역에 쓰기 시도

예를 들어, 코드 세그먼트는 읽기와 실행만 가능하도록 설정되어 있는데 어떤 프로세스가 그 영역에 값을 저장하려고 하면 이는 권한 위반이다.

- 주소 범위는 맞더라도 쓰기 권한이 없으므로 하드웨어가 예외를 발생시킨다.

(2) 실행 불가 영역에서 명령어 실행 시도

예를 드렁, 힙이나 스택은 데이터를 저장하는 공간이지, 원칙적으로 명령어를 실행하는 공간이 아니다.

그런데 프로그램이 실수로 혹은 악의적으로 힙이나 스택에 있는 데이터를 실행하려고 하면, 하드웨어는 이를 막아야한다.

- 주소는 유효할 수 있지만, 실행 권한이 없으므로 예외가 발생한다.

이처럼 protection bit 는 단순한 메모리 공유 지원뿐 아니라, 메모리 보호와 보안 강화에도 중요한 역할을 한다.

[4-7] 운영체제의 역할

하드웨어가 예외를 감지하면, 그다음 처리는 운영체제가 맡는다.

운영체제는 예외를 전달받아 다음과 같은 조치를 할 수 있다.

- 잘못된 접근을 한 프로세스를 종료
- 오류 메시지를 기록
- 메모리 보호 위반으로 간주하여 강제 중단
- 디버깅 환경이라면 예외 원인 분석 지원

즉, 하드웨어는 탐지, 운영체제는 처리를 담당한다.

세그먼트 공유가 안전하게 작동하려면 하드웨어만으로는 부족하고, 운영체제가 각 프로세스의 세그먼트 매핑과 권한 정보를 정확히 관리해야 한다.

[5] 소단위 대 대단위 세그멘테이션

지금까지 살펴본 세그멘테이션 예제들은 대부분 코드, 힙, 스택처럼 소수의 큰 세그먼트만 존재하는 구조를 가정하고 있었다.

이런 방식은 주소 공간을 몇 개의 큰 덩어리로 나누기 때문에 대단위 세그멘테이션(coarse-grained segmentation)이라고 부른다.

즉, 대단위 세그멘테이션에서는 주소 공간 전체를 세밀하게 쪼개기보다는, 프로그램 실행에 필요한 대표적인 논리 단위 몇 개만 구분한다.

예를 들면,

- 명령어가 저장된 코드 세그먼트
- 동적으로 할당되는 데이터를 위한 힙 세그먼트
- 함수 호출과 지역 변수 저장을 위한 스택 세그먼트

정도로 나누는 방식이다. 이 구조는 단순하고 이해하기 쉬우며, 필요한 하드웨어 구조도 비교적 간단하다.

각 세그먼트마다 베이스와 바운드 레지스터만 두면 되므로 구현이 직관적이다.

[5-1] 대단위 세그멘테이션의 특징

대단위 세그멘테이션은 말 그대로 큰 단위로 나누는 방식이다.

즉, 하나의 프로그램 주소 공간을 몇 개의 대표적인 큰 영역으로만 관리한다.

이 방식의 장점은 다음과 같다.

- | |
|--|
| <ul style="list-style-type: none">- 구조가 단순하다.- 하드웨어 구현이 쉽다.- 세그먼트 수가 적으므로 관리 비용이 작다.- 코드/힙/스택처럼 프로그램의 논리 구조와 잘 맞는다. |
|--|

하지만 단점도 있다.

세그먼트 하나가 비교적 크기 때문에, 그 내부에서 실제로 사용하지 않는 부분도 있어도 그 세그먼트 전체를 하나의 단위처럼 다뤄야 한다.

즉, 좀 더 세밀하게 관리할 수 있는 여지가 부족하다.

예를 들어, 어떤 프로그램의 코드 영역 안에서도 자주 쓰는 함수와 거의 쓰지 않는 함수가 섞여 있을 수 있고, 데이터 영역 안에서도 실제로는 일부만 활발히 사용될 수 있다. 그런데 대단위 세그멘테이션에서는 이런 차이를 세밀하게 반영하기 어렵다.

[5-2] 소단위 세그멘테이션이란?

이와 달리 일부 초기 시스템들은 주소 공간을 훨씬 더 잘게 나누는 방식을 채택하였다. 이를 소단위 세그멘테이션(fine-grained segmentation)이라고 한다.

소단위 세그멘테이션에서는 단순히 코드, 힙, 스택 정도로만 나누는 것이 아니라, 프로그램을 이루는 더 작은 논리 단위들을 각각 하나의 세그먼트로 볼 수 있다.

예를 들면,

- 함수 하나를 하나의 세그먼트로 나누거나
- 자료구조 단위로 세그먼트를 나누거나
- 모듈별로 코드를 분리하거나
- 데이터 집합별로 독립 세그먼트들을 둘 수 있다.

즉, 주소 공간을 훨씬 세밀하게 분해하여 관리하는 방식이다.

이렇게 하면 운영체제와 하드웨어는 프로그램을 구성하는 각 부분을 더 유연하게 다룰 수 있다.

필요할 때만 특정 세그먼트를 메모리에 두고, 사용하지 않는 세그먼트는 메모리에서 제외하거나 다른 저장장치로 내보내는 식의 세밀한 관리가 가능해진다.

[5-3] 왜 소단위 세그멘테이션이 필요했는가?

소단위 세그멘테이션이 등장한 배경에는 메모리를 더 효율적으로 쓰고 싶다는 목적이 있었다.

대단위 방식에서는 세그먼트 수가 적어서 단순하지만, 메모리를 세밀하게 제어하기 어렵다. 반면 소단위 방식에서는 프로그램이 여러 개의 작은 세그먼트로 나뉘므로, 운영체제는 그 중에서 현재 실제로 사용중인 부분만 메인 메모리에 유지할 수 있다.

예를 드렁, 프로그램 전체가 매우 크더라도, 지금 당장 필요한 함수 몇 개와 데이터 몇 개만 사용 중일 수 있다.

이런 경우 작은 세그먼트 단위로 나누어 두면, 사용 중인 세그먼트만 메모리에 두고 나머지는 제외할 수 있어 메모리 활용률이 높아진다.

즉, 소단위 세그멘테이션의 핵심 아이디어는 다음과 같다.

- 프로그램을 작은 논리 단위로 나누면, 운영체제가 필요한 부분만 더 정밀하게 관리할 수 있다.

[5-4] 많은 세그먼트를 어떻게 관리하는가?

그런데 세그먼트 수가 많아지면 문제가 생긴다.

대단위 세그멘테이션처럼 세그먼트가 3 개 정도라면 각 세그먼트에 대해 베이스/바운드 레지스터를 따로 두는 방식으로도 충분하다.

하지만 세그먼트가 수백 개, 수천 개가 되면 이런 방식은 불가능하다.

그래서 소단위 세그멘테이션을 지원하려면 세그먼트 테이블(segment table)이 필요하다.

세그먼트 테이블은 각 세그먼트의 정보를 메모리에 저장해 두는 자료구조이다.

각 항목에는 보통 다음과 같은 정보가 들어간다.

- | |
|--|
| <ul style="list-style-type: none">- 세그먼트의 시작 물리 주소(베이스)- 세그먼트의 길이(바운드)- 접근 권한 정보- 현재 사용 여부 또는 존재 여부 |
|--|

즉, 세그먼트 수가 많을 때는 레지스터 몇 개만으로 관리하지 않고,

메모리 안의 표(table)를 통해 각 세그먼트의 정보를 찾아 쓰는 방식으로 확장하는 것이다.

이 구조가 있으면 운영체제와 하드웨어는 수 많은 세그먼트를 유연하게 다룰 수 있게 된다.

[5-5] 역사적 예시의 의미

Multics, Burroughs B5000 같은 초기 시스템을 언급한다.

이 시스템들은 당시로서는 매우 진보적으로, 많은 수의 세그먼트를 지원했다.

예를 들어, Burroughs B5000 계열에서는 수천 개의 세그먼트를 다룰 수 있었고, 컴파일러가 프로그램의 코드와 데이터를 여러 개의 논리 단위로 잘게 분리해 내면, 운영체제와 하드웨어가 이 구조를 그대로 지원할 수 있었다.

이 말은 단순히 “세그먼트가 많았다”는 뜻이 아니다.

더 중요한 의미는 다음과 같다.

- | |
|--|
| <ul style="list-style-type: none">- 프로그램의 구조를 더 논리적으로 반영할 수 있었다.- 작은 단위로 보호와 공유를 설정할 수 있었다.- 사용 중인 부분과 사용하지 않는 부분을 구분하기 쉬웠다.- 메모리 적재와 회수가 더 유연해질 수 있었다. |
|--|

즉, 세그먼트가 많다는 것은 단지 개수가 늘어났다는 것이 아니라,

운영체제가 메모리를 훨씬 더 정교하게 제어할 수 있게 되었다는 뜻이다.

[5-6] 소단위 세그멘테이션의 장점

소단위 세그멘테이션은 다음과 같은 장점을 가진다.

첫째, 메모리 활용 효율이 높아질 수 있다.

프로그램 전체가 아니라 필요한 작은 부분만 메모리에 유지할 수 있기 때문이다.

둘째, 논리 구조를 더 잘 반영할 수 있다.

함수, 모듈, 데이터 구조별로 세그먼트를 나누면 프로그램의 구성과 메모리 관리 단위가 더 잘 대응한다.

셋째, 보호와 공유를 더 세밀하게 설정할 수 있다.

예를 들어, 특정 함수 코드만 공유하거나, 특정 데이터 세그먼트만 읽기 전용으로 둘 수 있다.

넷째, 사용 중인 세그먼트와 미사용 세그먼트를 구분하기 쉽다.

이는 운영체제가 메모리에 어떤 것을 남기고 어떤 것을 내보낼지 결정하는데 유리하다.

[5-7] 소단위 세그멘테이션의 부담

하지만 소단위 세그멘테이션이 항상 좋은 것만은 아니다.

세그먼트가 많아질수록 관리해야 할 정보도 많아진다.

즉,

- 세그먼트 테이블이 커진다.
- 주소 변환 과정이 더 복잡해진다.
- 운영체제가 추적해야 할 항목이 많아진다.
- 외부 단편화 문제가 더 심해질 가능성도 있다.

세그먼트 수가 많다는 것은 곧 유연성 증가와 동시에 관리 복잡도 증가를 의미한다.

그래서 대단위 세그멘테이션은 단순성과 구현 용이성에 장점이 있고,

소단위 세그멘테이션은 유연성과 효율성에서 장점이 있다고 볼 수 있다.

[5-8] 대단위와 소단위 비교

대단위 세그멘테이션

- 코드, 힙, 스택처럼 큰 단위로 나눔
- 세그먼트 수가 적음
- 구조가 단순함
- 관리가 쉬움
- 세밀한 제어는 어려움

소단위 세그멘테이션

- 함수, 모듈, 데이터 단위 등으로 잘게 나눔
- 세그먼트 수가 많음
- 세그먼트 테이블 필요
- 유연성이 높음
- 메모리를 더 세밀하게 관리 가능
- 관리 복잡도 커짐

[6] 운영체제의 지원

이제 세그멘테이션의 기본 개념은 이해할 수 있다. 세그멘테이션은 코드, 힙, 스택처럼 주소 공간의 각 부분을 서로 독립적인 세그먼트로 나누고, 각 세그먼트를 물리 메모리의 서로 다른 위치에 배치하는 방식이다.

이 방식의 가장 큰 장점은 프로세스 전체 주소 공간을 하나의 연속된 물리 공간에 통째로 올려둘 필요가 없다는 점이다.

예를 들어, 힙과 스택 사이에는 실제로 사용되지 않는 거대한 빈 영역이 존재할 수 있는데, 단순한 베이스-바운드 방식에서는 그 빈 영역까지 포함해서 물리 메모리를 확보해야 할 수도 있다.

반면 세그멘테이션에서는 코드, 힙, 스택만 각각 필요한 크기만큼 물리 메모리에 배치하면 되므로, 사용하지 않는 가상 주소 공간에 대해 물리 메모리를 낭비하지 않아도 된다.

그 결과, 같은 크기의 물리 메모리 안에 더 많은 프로세스를 동시에 적재할 수 있고, 메모리 활용 효율도 높아진다.

하지만 이러한 장점과 함께 운영체제는 새로운 책임을 지게 된다.

세그멘테이션을 제대로 지원하려면 운영체제가 단순히 프로세스를 실행시키는 수준을 넘어, 세그먼트의 위치, 크기, 보호 정보, 그리고 빈 물리 메모리 공간의 상태까지 정교하게 관리해야 한다.

[6-1] 문맥 교환 시 세그먼트 정보의 저장과 복원

첫 번째 문제는 문맥 교환이다.

프로세스 A가 실행되다가 중단되고, 이후 프로세스 B가 CPU를 넘겨받는 상황을 생각해보자.

이때 CPU 레지스터뿐 아니라 세그먼트 레지스터의 값도 함께 바뀌어야 한다.

왜냐하면 각 프로세스는 자신만의 가상 주소 공간을 가지며, 코드/힙/스택 세그먼트가 물리 메모리의 서로 다른 위치에 배치되어 있기 때문이다.

예를 들어, 프로세스 A의 코드 세그먼트는 물리 주소 32KB부터 시작할 수 있고, 프로세스 B의 코드 세그먼트는 물리 주소 80KB부터 시작할 수 있다.

만약 운영체제가 문맥 교환 시 세그먼트 레지스터를 제대로 바꾸지 않으면, 프로세스 B가 실행될 때도 여전히 A의 세그먼트 정보를 사용하게 된다.

그렇게 되면 B는 자신의 코드나 데이터가 아니라 다른 프로세스의 메모리 영역에 접근하는 심각한 오류를 일으킬 수 있다.

따라서 운영체제는 문맥 교환이 발생할 때마다 다음과 같은 작업을 수행해야 한다.

- 현재 실행 중인 프로세스의 세그먼트 레지스터 값 저장
- 다음에 실행할 프로세스의 세그먼트 레지스터 값 복원
- 각 세그먼트의 베이스, 바운드, 보호 비트 등의 정보 재설정

즉, 세그멘테이션 환경에서 문맥 교환은 단순한 CPU 상태 전환이 아니라, 주소 변환 환경 전체를 해당 프로세스에 맞게 다시 구성하는 과정이라 볼 수 있다.

[6-2] 가변 크기 세그먼트에 맞는 물리 메모리 관리

두 번째이자 더 중요한 문제는 빈 물리 메모리 공간을 어떻게 관리할 것인가이다.

기존의 단순한 메모리 관리 방식에서는 모든 프로세스가 비슷한 크기의 슬롯에 적재된다고 가정할 수 있었다.

예를 들어, 물리 메모리를 동일한 크기의 여러 칸으로 나누고, 빈 칸 하나에 프로세스 하나를 넣는 식이다.

그러나 세그멘테이션에서는 상황이 훨씬 복잡하다.

그 이유는 다음과 같다.

- 한 프로세스는 여러 개의 세그먼트를 가진다.
- 세그먼트마다 크기가 다르다.
- 힙은 실행 중에 커질 수 있다.
- 스택도 함수 호출이 많아지면 확장될 수 있다.
- 어떤 세그먼트는 종료되거나 해제되지만, 다른 세그먼트는 계속 남아 있을 수 있다.

즉, 물리 메모리에는 시간이 지나면서 크기가 제각각인 세그먼트들이 들어오고 나가게 된다. 운영체제는 새 세그먼트를 적재할 때마다 “이 크기의 세그먼트를 수용할 수 있는 연속된 빈 공간이 어디 있는가?”를 찾아야 한다.

문제는 이런 할당과 해제가 반복되면 물리 메모리 안의 작은 빈 공간들이 여기저기 흩어지게 된다는 점이다.

[6-3] 외부 단편화(external fragmentation)

이렇게 메모리 총량은 충분하지만, 연속된 큰 공간이 없어서 요청을 만족시키지 못한 현상을 외부 단편화라고 한다.

예를 들어 생각해보자.

- 어떤 시스템에 빈 공간이 총 24KB 남아 있다.
- 하지만 이 공간이 10KB, 8KB, 6KB 처럼 세 군데로 나누어져 있다.
- 이때 새로운 세그먼트가 20KB 를 요구하면, 총 빈 공간은 충분해도 하나의 연속된 20KB 공간이 없기 때문에 할당할 수 없다.

이것이 바로 외부 단편화의 핵심이다.

즉, “남의 메모리의 총합”보다 “연속된 빈 공간의 배치 상태”가 더 중요해진다.

세그멘테이션에서는 세그먼트 크기가 서로 다르고, 또 각 세그먼트가 물리 메모리의 아무 위치에 배치될 수 있으므로 이러한 문제가 특히 쉽게 발생한다.

시간이 지날수록 물리 메모리는 군데군데 잘게 쪼개진 빈 공간들로 채워지고, 결국 충분한 총 용량이 있어도 실제로 할당 가능한 공간은 부족해진다.

[6-4] 압축(compaction)

외부 단편화를 해결하는 대표적인 방법 중 하나는 압축이다.

압축은 흩어져 있는 세그먼트들을 한쪽으로 몰아서 재배치하고, 빈 공간을 한 덩어리로 큰 연속 영역으로 만드는 방법이다.

예를 들어, 메모리 안에 A, B, C 세그먼트가 띄엄띄엄 배치되어 있고 그 사이사이에 작은 빈 공간이 있다고 하자.

운영체제가 이 세그먼트들을 차례대로 앞으로 당겨서 붙여 놓으면, 뒤쪽에는 하나의 큰 빈공간이 남게 된다.

그렇게 되면 이전에 할당할 수 없었던 큰 세그먼트도 적재할 수 있게 된다.

하지만 압축은 현실적으로 비용이 매우 큰 작업이다. 이유는 다음과 같다.

- 이미 메모리에 올라와 있는 세그먼트의 내용을 실제로 복사해야 한다.
- 세그먼트를 옮긴 후에는 세그먼트 레지스터의 베이스 값도 다시 수정해야 한다.
- 실행 중인 프로세스를 일시 중단해야 할 수도 있다.
- 메모리 복사 자체가 많은 시간과 대역폭을 소모한다.

즉, 압축은 이론적으로는 단편화 해결에 효과적이지만, 실제로는 자주 수행하기 어려운 무거운 시스템 작업이다.

그래서 운영체제가 가능한 한 압축없이도 버틸 수 있도록 메모리 할당 정책을 잘 설계하려고 한다.

[6-5] 빈 공간 관리 알고리즘

압축이 너무 비싸기 때문에 운영체제는 보통 빈 공간 리스트(free list)를 관리하면서 세그먼트를 배치한다.

빈 공간 리스트란 현재 할당되지 않은 물리 메모리 영역들을 따로 기록해 둔 자료구조이다. 새 세그먼트가 들어오면 운영체제는 이 리스트를 검사하여 적절한 빈 공간을 찾아 할당한다.

대표적인 방식들은 다음과 같다.

최초 적합(first-fit)

리스트를 처음부터 순서대로 살펴보면서, 요청한 크기 이상인 첫 번째 빈 공간에 할당하는 방법이다.

구현이 단순하고 빠르지만, 앞부분의 작은 빈 공간들이 자주 잘게 나뉘어 단편화가 심해질 수 있다.

최적 적합(best-fit)

요청 크기와 가장 비슷한, 즉 딱 맞거나 가장 조금 남는 빈 공간을 선택하는 방식이다.

겉보기에는 낭비를 줄일 것 같지만, 실제로는 너무 작은 자투리 공간들을 많이 만들어내는 경우가 있다.

최악 적합(worst-fit)

가장 큰 빈 공간을 선택해서 그 일부를 할당하는 방식이다.

작은 조각이 생기는 것을 줄이려는 생각에서 나온 방법이지만, 실제 효과는 상황에 따라 다르다.

버디 알고리즘(buddy algorithm)

메모리를 일정한 규칙에 따라 2의 거듭제곱 크기로 분할하고 합치는 방식이다.

할당과 병합이 비교적 빠르고 체계적이지만, 요청 크기와 정확히 일치하지 않을 경우 내부 낭비가 생길 수 있다.

이러한 알고리즘들은 모두 “빈 공간을 어떻게 고를 것인가”에 대한 전략 차이일뿐, 공통 목표는 하나이다.

바로 외부 단편화를 가능한 줄이면서도 빠르게 할당과 해제를 처리하는 것이다.

[6-6] 그러나 단편화 문제는 완전히 사라지지 않는다.

중요한 점은, 어떤 알고리즘을 사용하더라도 외부 단편화를 완전히 없애기는 어렵다는 것이다.

세그멘테이션은 본질적으로 크기가 서로 다른 가변 길이 블록들을 연속 공간에 배치하는 방식이기 때문에, 시간이 지나면 메모리가 조각나는 현상이 자연스럽게 발생한다.

즉, 운영체제가 할 수 있는 일은 단편화를 “완전히 제거”하는 것이 아니라.

- 단편화가 덜 심해질도록 잘 배치하고,
- 빈 공간을 효율적으로 추적하고,
- 필요할 때만 제한적으로 압축하고,
- 전체 시스템 성능과 메모리 사용률 사이에서 균형을 맞추는 것

이라고 볼 수 있다.

[1-4] 가상화: 빈 공간 관리

핵심 질문: 빈 공간을 어떻게 관리하는가

가변 크기의 요구를 충족시켜야 할 때, 빈 공간은 어떻게 관리되어야 하는가?

단편화를 최소화하기 위해 어떤 전략을 사용할 수 있는가?

여러 대안들의 시간과 공간의 오버헤드는 어떻게 되는가?

이 장에서는 메모리 가상화에 대한 논의를 잠시 벗어나, 모든 메모리 관리 시스템에서 공통적으로 등장하는 근본적인 문제를 다룬다.

여기서 말하는 메모리 관리 시스템은 두 가지 형태가 될 수 있다.

하나는 프로세스의 힙 영역을 관리하는 malloc 과 같은 사용자 수준의 메모리 할당 라이브러리이고, 다른 하나는 프로세스 주소 공간의 일부를 관리하는 운영체제의 메모리 관리 기능이다.

이 장에서 집중적으로 다룰 주제는 빈 공간(free space)을 어떻게 관리할 것인가라는 문제이다.

먼저 문제를 단순한 경우부터 생각해 볼 수 있다.

예를 들어, 관리해야 하는 메모리가 모두 동일한 크기의 블록으로 나누어져 있는 경우에는 빈 공간 관리가 비교적 쉽다.

이러한 경우에는 단순한 고정 크기 블록들의 리스트를 유지하면 된다.

프로그램이 메모리를 요청하면 리스트에서 첫 번째 블록을 꺼내어 할당하고, 사용이 끝난 블록은 다시 리스트에 반환하면 된다.

이런 방식에서 블록의 크기가 항상 동일하기 때문에 관리가 단순하며, 복잡한 공간 탐색 과정도 필요하지 않다.

그러나 실제 시스템에서는 상황이 훨씬 복잡해진다.

가변 크기의 메모리 공간을 관리해야 하는 경우에는 빈 공간 관리가 훨씬 어려워지고 흥미로운 문제가 발생한다.

이러한 상황은 대표적으로 두 곳에서 나타난다.

하나는 malloc()과 free()를 사용하면 사용자 수준의 메모리 할당 라이브러리이고, 다른 하나는 세그멘테이션 기법을 이용하여 물리 메모리를 관리하는 운영체제이다.

이 경우에는 메모리의 요청의 크기가 매번 달라지기 때문에 빈 공간도 다양한 크기로 남게 된다.

이러한 환경에서는 외부 단편화(external fragmentation)라는 문제가 발생한다.

메모리가 반복적으로 할당되고 해제되는 과정에서, 빈 공간이 서로 다른 크기의 작은 조각들로 나누어지게 된다.

시간이 지날수록 이러한 조각들이 메모리 곳곳에 흩어지게 되고, 그 결과 전체 빈 공간의 총합은 충분하지만 하나의 연속된 큰 공간이 존재하지 않는 상황이 발생할 수 있다.

예를 들어, 메모리에는 총 20 바이트의 빈 공간이 존재하지만, 이 공간이 10 바이트 짜리 두 개의 조각으로 나누어져 있다. 이때 프로그램이 15 바이트의 연속된 공간을 요청한다면, 전체 빈 공간의 크기는 충분함에도 불구하고 요청을 만족시킬 수 없다. 왜냐하면 요청된 크기를 담을 수 있는 연속된 메모리 영역이 존재하지 않기 때문이다.

이와 같은 문제는 메모리 관리 시스템에서 매우 중요한 이슈이며, 바로 이러한 외부 단편화 문제를 어떻게 해결하거나 완화할 것인가 이 장에서 다루고자 하는 핵심 주제이다.

[1] 가정

이 장에서는 주로 사용자 수준 메모리 할당 라이브러리에서 사용되는 메모리 할당기(memory allocator)의 발전 과정과 동작 방식을 중심으로 설명한다.

이러한 내용은 Wilson 등의 연구에서 잘 정리되어 있으며, 보다 자세한 내용을 알고 싶다면 해당 문헌을 참고하는 것이 좋다.

이 장에서는 기본적으로 malloc()과 free()가 제공하는 메모리 할당 인터페이스를 가정한다. 먼저 malloc()함수는 다음과 같은 형태를 가진다.

```
void *malloc(size_t size)
```

이 함수는 응용 프로그램이 필요로 하는 메모리의 크기를 size 라는 변수로 전달받는다.

size 는 요청된 바이트 수를 의미하며, 메모리 할당기는 이 요청을 처리한 뒤 요청된 크기와 같거나 그보다 큰 메모리 영역이 시작 주소를 반환한다.

반환되는 값의 타입은 특정 자료형이 아닌 void* 포인터이며, 이는 C 언어에서 어떤 데이터 타입의 데이터도 가리킬 수 있는 포인터를 의미한다.

프로그램은 이후 이 포인터를 자신이 원하는 자료형 포인터로 변환하여 사용할 수 있다.

메모리 해제를 위한 함수는 다음과 같다.

```
void free(void *ptr)
```

free()함수는 malloc()을 통해 할당받았던 메모리 영역을 다시 반환할 때 사용된다.

이 함수는 해제할 메모리 영역의 시작 주소를 가리키는 포인터 ptr 만을 인자로 받는다.

여기서 중요한 점은, 메모리를 해제할 때 사용자가 메모리 크기 정보를 전달하지 않는다는 것이다.

즉, free()는 단순한 포인터만 전달받는다.

따라서 메모리 할당 라이브러리는 포인터만을 이용해 해당 메모리 영역의 크기를 알아내야 한다.

이러한 정보를 어떻게 관리하고 추적하는지는 이후에 자세히 설명될 것이다.

메모리 할당 라이브러리가 관리하는 공간은 전통적으로 힙(heap)이라고 불린다.

힙은 프로그램 실행 중 동적으로 메모리를 할당하고 해제하는 영역이며, 메모리 할당기는 이 힙 공간에서 사용 가능한 빈 공간을 관리해야 한다.

일반적으로 힙의 빈 공간을 관리하기 위해 링크드 리스트(linked list)와 같은 자료구조가 사용된다.

이 리스트에는 현재 할당되지 않은 빈 메모리 블록(또는 청크, chunk)들의 위치 정보가 저장된다.

물론 반드시 링크드 리스트만 사용할 필요는 없으며, 빈 공간들을 효율적으로 표현하고 관리할 수 있는 다른 자료구조를 사용할 수 있다.

이 장에서는 특히 외부 단편화 문제에 중점을 두어 설명할 것이다.

외부 단편화란 메모리 할당과 해제가 반복되는 과정에서 빈 공간이 여러 개의 작은 조각들로 나누어지는 현상을 의미한다.

시간이 지나면서 이러한 작은 조각들이 메모리 곳곳에 흩어지게 되면, 전체 빈 공간의 총합은 충분하더라도 요청된 크기의 연속된 공간을 찾지 못하는 상황이 발생할 수 있다.

물론 메모리 관리에서는 내부 단편화(internal fragmentation) 문제도 발생할 수 있다.

내부 단편화는 메모리 할당과 요청된 크기보다 더 큰 메모리 블록을 할당할 때 발생한다.

이 경우 실제로 프로그램이 사용하지 않는 공간이 할당된 메모리 블록 내부에 남게 되며, 그 공간은 낭비된 것으로 간주된다.

즉, 내부 단편화 역시 메모리 낭비의 한 형태라고 볼 수 있다.

그러나 이 장에서는 논의를 단순화하기 위해, 그리고 메모리 관리에서 더 흥미롭고 중요한 문제이기 때문에 외부 단편화 문제를 중심으로 설명한다.

또한 이장에서는 이미 할당된 메모리는 다른 위치로 이동시킬 수 없다고 가정한다.

예를 들어, 프로그램이 malloc()을 호출하여 힙 영역의 일부 메모리에 대한 포인터를 받았다면, 해당 메모리는 free()를 통해 해제되기 전까지 프로그램이 계속 사용할 수 있어야 한다.

따라서 메모리 할당 라이브러리는 프로그램이 사용하는 동안 그 메모리를 다른 위치로 이동시킬 수 없다.

이러한 가정 때문에 메모리 압축(compaction)과 같은 방법을 사용할 수 없다.

메모리 압축은 흩어진 메모리 블록들을 이동시켜 빈 공간을 하나의 큰 연속 영역으로 만드는 방법인데, 이미 프로그램이 사용 중인 메모리를 이동시키면 포인터 값이 달라져 프로그램이 정상적으로 동작하지 않게 된다.

따라서 사용자 수준의 메모리 할당 라이브러리에서는 이러한 방식의 단편화 해결 방법을 적용하기 어렵다.

반면, 운영체제가 세그멘테이션 방식으로 메모리를 관리하는 경우에는 상황이 조금 다르다.

운영체제는 세그먼트의 물리적 위치를 변경하고 관련 레지스터 값을 수정할 수 있기 때문에, 단편화 문제를 해결하기 위해 메모리 압축 기법을 사용할 수 있다.

이러한 내용을 세그멘테이션을 설명하는 장에서 이미 언급한 바 있다.

[2] 저수준 기법들

이 절에서는 구체적인 메모리 할당 정책을 설명하기 전에, 대부분의 메모리 할당기에서 공통적으로 사용되는 기본적인 저수준 기법들을 먼저 살펴본다.

이러한 기법들은 메모리 관리의 핵심 동작을 이해하는데 매우 중요하다.

여기서는 크게 세 가지를 설명한다.

1. 분할(splitting)과 병합(coalescing)
2. 할당된 메모리 블록의 크기를 알아내는 방법
3. 빈 공간 리스트를 실제 메모리 내부에 구현하는 방법

[2-1] 분할(Splitting)과 병합(Coalescing)

먼저 빈 공간 리스트(free list)는 힙(heap) 안에서 아직 사용되지 않은 메모리 영역들의 집합을 의미한다.

예를 들어 총 30 바이트의 힙이 있다고 가정하자.

이 힙에는 다음과 같은 상태가 존재할 수 있다.

- 바이트 0 ~ 9: 빈 공간(10 바이트)
- 바이트 10 ~ 19: 사용 중
- 바이트 20 ~ 29: 빈 공간(10 바이트)

따라서 빈 공간 리스트에는 두 개의 빈 청크(chunk)가 존재한다.

이 상태에서 10 바이트보다 큰 요청이 들어오면, 요청을 충족할 수 있는 연속된 빈 공간이 없기 때문에 할당은 실패하고 NULL 을 반환한다.

하지만 10 바이트보다 작은 요청이 들어오면 상황이 달라진다.

예를 들어, 프로그램이 1 바이트를 요청했다고 가정하자.

이 경우 메모리 할당기는 분할(splitting)이라는 작업을 수행한다.

분할이란 하나의 빈 메모리 블록을 두 개로 나누는 작업이다.

과정은 다음과 같다.

1. 요청을 만족시킬 수 있는 빈 청크를 찾는다.
2. 해당 청크를 요청된 크기 + 남는 공간으로 나눈다.
3. 앞부분은 사용자에게 할당한다.
4. 나머지 공간은 다시 빈 공간 리스트에 남겨 둔다.

예를 들어, 두 번째 빈 공간(20~29)을 사용한다고 하면

- 20 번 주소: 사용자에게 1 바이트 할당
- 21~29: 9 바이트 빈 공간

즉 결과적으로

- 새로운 빈 공간 시작 주소: 21
- 새로운 빈 공간 크기: 9 바이트

가 된다.

이처럼 요청 크기가 빈 청크보다 작은 경우 분할 기법이 사용된다.

병합(Coalescing)

분할과 함께 반드시 등장하는 개념이 병합이다.

다시 다음과 같은 상태를 생각해보자.

- 빈 공간 10 바이트
- 사용 중 10 바이트
- 빈 공간 10 바이트

여기서 프로그램이

```
free(10)
```

을 호출하여 가운데 사용 중이던 10 바이트를 해제했다고 하자.

단순하게 생각하면 이 공간을 그냥 빈 공간 리스트에 추가할 것이다.

그러면 메모리는 다음과 같은 상태가 된다.

- 빈 공간 10
- 빈 공간 10
- 빈 공간 10

즉, 총 30 바이트가 비어 있다.

하지만 문제는 각각 10 바이트 조각이라는 점이다.

이 상태에서 20 바이트 요청이 들어오면

실제로는 메모리가 충분하지만 연속된 20 바이트 공간이 없기 때문에 할당이 실패하게 된다.

이 문제를 해결하기 위해 사용하는 방법이 병합이다.

병합의 원리는 매우 간단하다.

메모리 블록의 해제될 때

1. 해제된 블록의 앞 주소
2. 해제된 블록의 뒤 주소

를 확인한다.

만약 이웃한 블록이 빈 공간이라면

- 두 블록을 하나의 큰 블록으로 합친다.

예를 들어,

[빈 10] [빈 10] [빈 10]

이 세 블록은 병합 후

[빈 30]

이 된다.

이렇게 하면 큰 메모리 요청도 처리할 수 있게 된다.

[2-2] 할당된 공간의 크기 파악

free()함수의 인터페이스를 다시 보면 다음과 같다.

```
void free(void *ptr)
```

여기서 중요한 특징은 메모리 크기를 전달하지 않는다는 것이다.

즉, free()는 단순히 포인터만 전달받는다.

하지만, 메모리를 해제하려면 다음 정보를 알아야 한다.

- 이 블록의 크기
- 이 블록의 경계

그래야 빈 공간 리스트에 다시 추가할 수 있기 때문이다.

이 문제를 해결하기 위해 대부분의 메모리 할당기는 헤더(header)를 사용한다.

헤더는 사용자에게 반환되는 메모리 블록 바로 앞에 저장되는 추가 정보 영역이다.

예를 들어,

[헤더][사용자 데이터]

형태로 메모리가 배치된다.

헤더에는 보통 다음 정보가 저장된다.

- 메모리 블록 크기
- 무결성 검사용 값(magic number)
- 포인터 등 추가 정보

예를 들어, 다음과 같은 구조체를 사용할 수 있다.

```
typedef struct __header_t {  
    int size;  
    int magic;  
} header_t;
```

- size: 할당된 메모리 크기
- magic: 오류 검사를 위한 값

사용자가

```
ptr = malloc(20);
```

을 호출하면 메모리 구조는 다음과 같다.

```
[header][20 바이트 데이터]
```

사용자가

```
free(ptr)
```

을 호출하면 라이브러리는 포인터 연산을 통해 헤더 위치를 계산한다.

예시코드

```
header_t *hptr = (void *)ptr - sizeof(header_t);
```

- 헤더 주소를 얻을 수 있고
- size 값을 읽어
- 정확한 메모리 크기를 알 수 있다.

[2-3] 빈 공간 리스트 내장

지금까지 우리는 빈 공간 리스트라는 개념을 사용해 왔다.
하지만 중요한 질문이 하나 있다.

이 리스트는 어디에 저장되는가?

일반적으로 프로그램에서 새로운 리스트 노드를 만들 때는 malloc()을 사용한다.
하지만 메모리 할당 라이브러리 내부에서는 malloc을 사용할 수 없다.

그래서 해결 방법은 빈 공간 자체 안에 리스트 노드를 저장하는 것이다.
예를 들어, 다음과 같은 노드 구조를 사용할 수 있다.

```
typedef struct __node_t {  
    int size;  
    struct __node_t *next;  
} node_t;
```

힙 초기화

예를 들어, 4KB 힙을 초기화한다고 가정하자.

```
node_t *head = mmap(NULL, 4096,  
    PROT_READ | PROT_WRITE,  
    MAP_ANON | MAP_PRIVATE,  
    -1, 0);
```

이 코드는 운영체제로부터 4096 바이트 메모리 영역을 할당받는다.
그리고 초기 상태는 다음과 같다.

```
head->size = 4096 - sizeof(node_t);  
head->next = NULL;
```

- 하나의 빈 공간
- 크기 = 약 4088 바이트

메모리 할당 과정 예시

이제 프로그램이 100 바이트를 요청했다고 가정하자.

할당 과정

1. 충분히 큰 빈 공간 탐색
2. 청크 분할 수행
3. 헤더 생성
4. 사용자에게 포인터 반환

예를 들어, 헤더 크기 = 8 바이트라면 총 필요한 공간은

$100 + 8 = 108$ 바이트

free() 호출시 동작

프로그램이 free()를 호출하면

1. 헤더를 통해 블록 크기 확인
2. 빈 공간 리스트에 삽입
3. 인접한 블록이 있으면 병합 수행

병합이 없다면 다음과 같은 문제가 생긴다.

[빈][빈][빈][빈]

즉, 메모리가 조각나 있는 상태가 된다.

병합을 수행하면

[큰 빈 공간]

으로 다시 합쳐진다.

힙 확장

마지막으로 힙 공간이 부족한 경우를 생각해 보자.

가장 단순한 방법은

malloc() -> NULL 반환

이다.

하지만 대부분의 메모리 할당기는 힙을 확장하는 방법을 사용한다.

대표적인 방법은 sbrk() 시스템 호출이다.

과정

1. 할당기가 sbrk() 호출
2. 운영체제가 새로운 메모리 페이지 할당
3. 프로세스 주소 공간에 매핑
4. 힙 크기 증가

이렇게 하면 프로그램은 더 큰 힙 공간을 사용할 수 있게 된다.

[3] 기본 전략

앞 절에서 메모리 할당기의 저수준 기법(분할, 병합 등)을 살펴보았으므로, 이제는 빈 공간을 실제로 어떻게 선택할 것인가에 대한 기본 전략을 살펴본다.

이러한 전략들은 복잡해 보이지만, 사실은 직관적으로 생각할 수 있는 간단한 아이디어들이다.

하지만 중요한 점은, 어떤 전략도 항상 좋은 성능을 보장하지는 않는다는 것이다. 이유는 다음과 같다.

- 메모리 패턴이 무작위적이다.
- 요청은 프로그래머에 의해 결정된다.
- 특정 전략은 어떤 입력에서는 좋지만, 다른 입력에서는 매우 비효율적일 수 있다.

따라서 목표는 하나의 완벽한 전략을 찾는 것이 아니라, 각 전략의 특징과 장단점을 이해하는 것이다.

이상적인 메모리 할당기는 다음 두 가지를 동시에 만족해야 한다.

- 빠른 속도
- 낮은 단편화

하지만 현실에서는 이 두가지가 트레이드오프 관계에 있다.

[3-1] 최적 적합(Best Fit)

최적 적합은 이름 그대로 가장 “딱 맞는” 빈 공간을 선택하는 전략이다.

동작 방식

1. 빈 공간 리스트 전체를 탐색한다.
2. 요청 크기 이상인 블록들을 찾는다.
3. 그 중에서 가장 작은 블록을 선택한다.

즉, “남는 공간이 최소가 되도록” 선택하는 방식이다.

의도

- 메모리 낭비를 최소화하려는 목적
- 내부 단편화를 줄이려는 시도

장점

- 비교적 공간 효율이 좋다.
- 큰 블록을 불필요하게 소비하지 않는다.

단점

- 항상 전체 리스트를 탐색해야한다.(느림)
- 결과적으로 작은 조각들이 많이 남는다.(외부 단편화 증가)

[3-2] 최악 적합(Worst Fit)

최악 적합은 최적 적합과 반대로, 가장 큰 빈 공간을 선택하는 전략이다.

동작 방식

- | |
|--|
| <ol style="list-style-type: none">1. 빈 공간 리스트 전체 탐색2. 가장 큰 블록 선택3. 요청 크기만큼 사용하고 나머지 유지 |
|--|

의도

- 작은 조각 대신 큰 공간을 유지하려는 전략

장점

- 작은 단편이 생기는 것을 줄이려는 시도

단점

- 역시 전체 탐색 필요(느림)
- 실제로는 단편화가 많이 발생하는 경우가 많음
- 연구 결과에서도 효율이 좋지 않음

[3-3] 최초 적합(First Fit)

최초 적합은 가장 단순하고 많이 사용되는 방식이다.

동작 방식

- | |
|---|
| <ol style="list-style-type: none">1. 리스트를 처음부터 탐색2. 요청 크기 이상인 첫 번째 블록을 찾으면 즉시 할당 |
|---|

장점

- 빠르다(전체 탐색 불필요)
- 구현이 간단하다

단점

- 리스트 앞부분에 작은 조각들이 몰림
- 시간이 지나면 초반부 단편화 심화

개선 방법

- 주소 기반 정렬(address-based ordering) 사용
 - 주소 순으로 정렬
 - 병합 쉬워짐
 - 단편화 감소

[3-4] 다음 적합(Next Fit)

다음 적합은 최초 적합을 개선하는 방식이다.

차이점

- 항상 리스트 처음부터 시작하지 않음
- 마지막으로 탐색했던 위치부터 다시 시작
- 즉, 내부적으로 포인터 하나를 유지한다.

장점

- 탐색이 리스트 전체에 고르게 분산됨
- 앞부분에 단편화 집중되는 문제 완화
- 성능은 최초 적합과 비슷

단점

- 최적 해를 보장하지 않음
- 여전히 단편화 문제 존재

[4] 다른 접근법

앞에서 살펴본 기본적인 메모리 할당 방식들 외에도, 메모리 할당의 효율을 높이기 위한 다양한 기술과 알고리즘이 제안되어 왔다.

여기서는 그중 몇 가지를 소개한다. 단순히 최적 적합과 같은 기본 전략에 머무르지 않고, 그보다 더 발전된 방법들을 이해해 보는 것이 목적이다.

[4-1] 개별 리스트(Segregated List)

먼저, 한동안 널리 사용되었던 방법으로 개별 리스트 방식이 있다.

이 방법의 기본 아이디어는 비교적 간단하다.

어떤 응용 프로그램이 특정 크기의 메모리를 자주 요청하는 경우, 그 크기의 블록들만을 따로 관리하는 별도의 리스트를 유지하는 것이다.

그리고 그 외의 다양한 크기의 요청들은 일반적인 메모리 할당기를 통해 처리하도록 한다.

이 방식의 장점은 명확하다.

특정 크기의 메모리 블록들을 미리 따로 관리하기 때문에 단편화가 줄어들고, 원하는 크기의 블록을 빠르게 찾을 수 있어 할당과 해제 과정이 매우 효율적으로 이루어진다.

특히 복잡하나 탐색 과정이 필요 없다는 점에서 성능상 이점이 크다.

하지만 이러한 방식도 새로운 문제를 발생시킨다.

예를 들어, 특정 크기의 메모리 풀과 일반적인 메모리 풀에 각각 얼마만큼의 메모리를 할당해야 하는지가 중요한 문제가 된다.

특정 크기에 너무 많은 메모리를 할당하면 낭비가 발생할 수 있고, 반대로 부족하면 다시 일반 할당기를 사용해야 하는 비효율이 생긴다.

[4-2] 슬랩 할당기(Slab Allocator)

이 문제를 더 효과적으로 해결하기 위해 등장한 방식이 슬랩 할당기이다.

이 방식은 커널에서 자주 사용되는 객체들을 효율적으로 관리하는 데 초점을 맞추고 있다.

시스템이 부팅될 때, 커널은 락이나 파일 시스템의 아이노드와 같은 자주 사용되는 자료 구조들을 위한 여러 개의 객체 캐시를 미리 생성한다.

각 객체 캐시는 동일한 크기의 객체들로 이루어진 빈 공간 리스트이며, 메모리 할당과 해제를 바르게 처리하기 위해 사용된다.

만약 기존에 할당된 캐시 공간이 부족해지면, 상위 메모리 할당기로부터 새로운 메모리 블록, 즉 슬랩을 추가로 요청한다.

이때 요청되는 메모리는 보통 페이지 크기의 정수배 단위이다.

반대로, 특정 슬랩에 포함된 객체들이 더 이상 사용되지 않으면, 해당 슬랩은 다시 상위 할당기로 반환될 수 있다.

슬랩 할당 방식의 중요한 특징은 객체를 미리 초기화된 상태로 유지한다는 점이다.

일반적인 할당 방식에서는 객체를 사용할 때마다 초기화하고, 반환할때 정리하는 과정이 반복되는, 이 과정은 상당한 비용을 발생시킨다.

하지만 슬랩 할당기에서는 한 번 초기화된 객체를 계속 재사용할 수 있도록 유지함으로써 이러한 오버헤드를 크게 줄일 수 있다.

[3-3] 버디 할당(Buddy Allocator)

다음으로, 빈 공간을 효율적으로 병합하기 위한 방법으로 버디 할당 방식이 있다.

이 방식에서는 메모리를 처음에 하나의 큰 공간, 즉 크기의 2^n 의 블록으로 간주하고, 메모리 요청이 발생하면 필요한 크기에 도달할 때까지 공간을 반으로 계속 분할한다.

더 이상 나눌 수 없을 정도로 적절한 크기가 되면 해당 블록을 사용자에게 할당한다.

예를 들어, 64KB의 빈 공간에서 7KB를 요청하면, 공간을 계속 반으로 나누어 결국 8KB 크기의 블록을 할당하게 된다.

이 방식은 항상 2의 거듭제곱 크기의 블록만을 사용하기 때문에 내부 단편화가 발생할 수 있다는 단점이 있다.

그러나 버디 할당의 가장 큰 장점은 메모리 해제 시에 나타났다.

어떤 블록이 반환되면, 할당기는 해당 블록의 “버디”가 비어 있는지를 확인한다.

만약 버디 블록도 비어 있다면, 두 블록을 하나로 합쳐 더 큰 블록으로 만든다.

이러한 병합 과정은 상위 단계로 계속 반복되며, 더 이상 병합할 수 없을 때까지 진행된다.

이 방식이 효율적인 이유는 특정 블록의 바디를 쉽게 찾을 수 있기 때문이다.

버디 관계에 있는 두 블록은 주소 상에서 단 하나의 비트만 다르기 때문에, 간단한 연산으로 빠르게 확인할 수 있다.

[3-4] 기타 접근법

마지막으로, 지금까지 설명한 방법들의 공통적인 문제 중 하나는 확장성이다.

빈 공간의 개수가 많아질수록 리스트를 탐색하는 비용이 증가하여 성능이 저하될 수 있다.

이를 해결하기 위해 보다 복잡한 자료구조를 사용하는 방법들이 제안되었다.

예를 들어, 균형 이진 트리나 스펙트럼 트리나 같은 구조를 활용하면 탐색 시간을 줄일 수 있다.

하지만 이러한 방법들은 구현이 복잡해진다는 단점이 있다.

또한, 현대의 시스템은 대부분 멀티프로세서 및 멀티스레드 환경에서 동작하기 때문에, 이러한 환경에 맞게 메모리 할당기를 최적화하는 연구도 활발히 진행되고 있다.

여러 스레드가 동시에 메모리를 할당하고 해제하는 상황에서 성능 저하를 방지하기 위한 다양한 기법들이 필요하다.

이처럼 메모리 할당 문제는 단순한 전략만으로 해결되기 어려우며, 다양한 상황을 고려한 여러 기법들이 함께 사용된다.